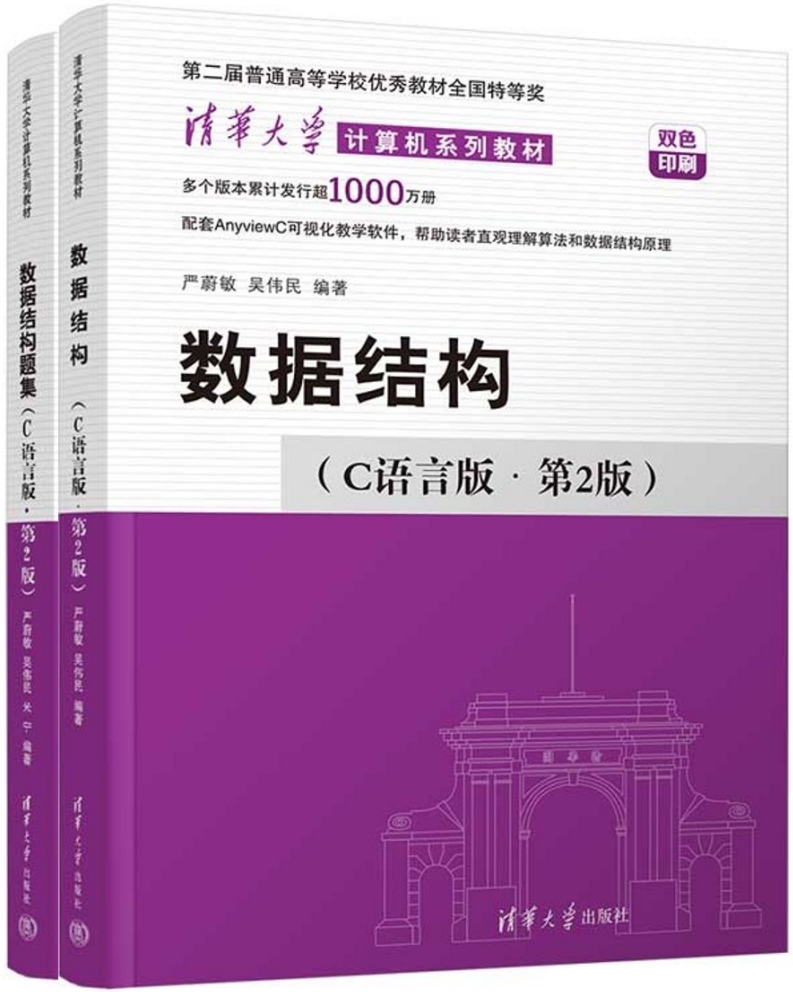


清華大学出版社
TSINGHUA UNIVERSITY PRESS

第6章

树和二叉树

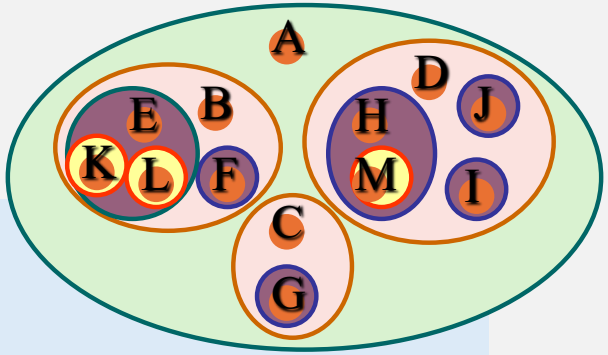
6.1 树的定义和基本术语

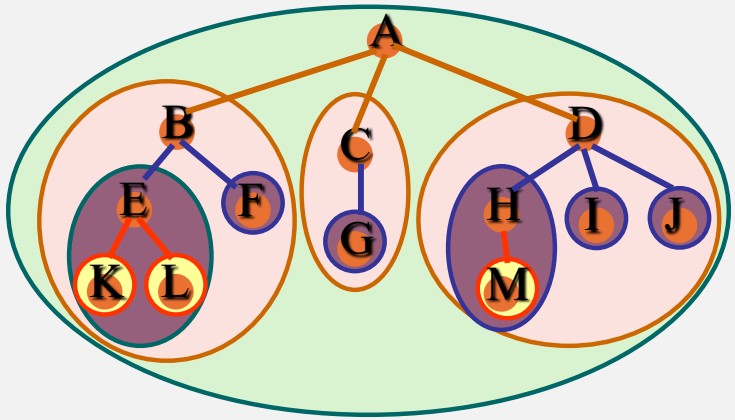
清華大學出版社 TSINGHUA UNIVERSITY PRESS

树的抽象数据类型

数据对象 D：

D是具有相同特性的数据元素的集合。



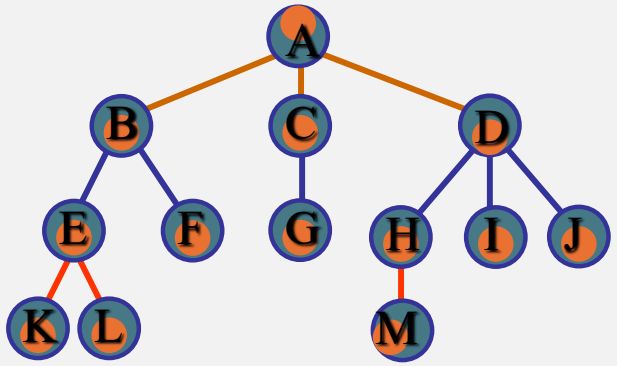


数据关系 R：

若D为空集，
则称为空树；
否则：

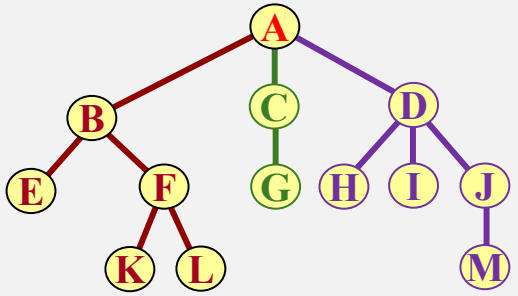
(1) 在D中存在唯一的称为根的数据元素root，

(2) 当n>1时，其余结点可分为m (m>0)个互不相交的有限集$T_1, T_2, \ldots, T_m$，其中每一个子集本身又是一棵符合本定义的树，称为根root的子树。



基本操作：

```
InitTree();                 // 构造并返回空树
MakeTree(e, n, ...);        // 组建并返回一棵树
FreeTree(T);                // 回收树T
TreeEmpty(T);               // 判空树
TreeDepth(T);               // 返回树T的深度
Root(T);                    // 返回树T的根
Value(T, node);             // 返回node节点的值
Assign(T, node, value);     // node节点赋值value
Parent(T, node);            // 返回node节点的双亲
FirstChild(T, node);        // 返回node节点的第一个孩子
NextSib(T, node);           // 返回node节点的下一个兄弟
InsertSubTree(T, i, Ti);    // 插入Ti为T节点的第i棵子树
DeleteSubTree(T, i);        // 删除并返回T节点的第i棵子树
TravTree(T, Visit());       // 按某种次序遍历树T
```



树的广义表表示：

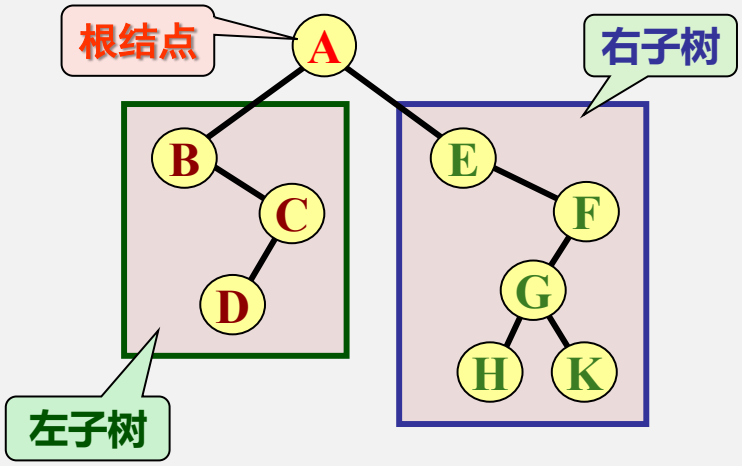
A(B(E, F(K, L)), C(G), D(H, I, J(M)))

树根　T_1　T_2　T_3

这是将树结构序列化的一种方式

6.2 二叉树

二叉树或为空树；
或是由一个根结点加上两棵分别称为
左子树和右子树的、互不交的二叉树组成。



广义表形式：A(B(#, C(D, #)), E(#,F(G(H,K),#)))

其中的 # 表示空子树，以区分对应的非空子树的“左右”

递归定义表明：二叉树或为空，或是由一个根节点加上两棵分别称为左子树和右子树的、互不相交的二叉树组成。

由于这两棵子树也是二叉树，则由二叉树的定义，它们也可以是空树。

由此，二叉树可以有**5种基本形态**，如图6.3所示。

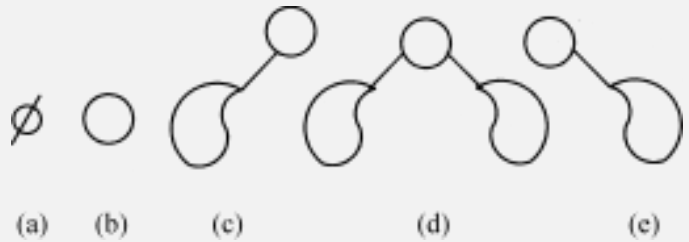


图6. 3 二叉树的5种基本形态

(a）空二叉树；(b）仅有根节点的二叉树；(c）右子树为空的二叉树；(d）左、右子树均非空的二叉树；(e）左子树为空的二叉树

二叉树的基本操作

清華大學出版社
TSINGHUA UNIVERSITY PRESS

基本操作：

```
InitBiTree();                 // 构造并返回一棵空二叉树
MakeBiTree(e, L, R);          // 组建并返回一棵二叉树
FreeBiTree(T);                // 回收二叉树T的全部节点
BiTreeEmpty(T);               // 判空
BiTreeDepth(T);               // 返回二叉树T的深度
Root(T);                      // 返回二叉树T的根
Value(T, node);               // 返回node节点的值
Assign(T, node, value);       // 对node节点赋值value
Parent(T, node);              // 返回node节点的双亲
LeftChild(T, node);           // 返回node节点的左孩子
RightChild(T, node);          // 返回node节点的右孩子
ReplaceLeft(T, LT);           // 用LT替换T节点的左子树
ReplaceRight(T, RT);          // 用RT替换T节点的右子树
PreOrderTrav(T, Visit());     // 先序遍历二叉树T
InOrderTrav(T,Visit());       // 中序遍历二叉树T
PostOrderTrav(T, Visit());    // 后序遍历二叉树T
LevelOrderTrav(T, Visit()); // 层序遍历二叉树T
```

2026/10/2

二叉树的性质

性质1 在二叉树的第i层上至多有2^{i-1}个节点($i \geq 1$)。

性质2 深度为k的二叉树至多有2^k-1个节点($k \geq 1$)。

性质3 对任何一棵二叉树T，如果其终端节点数为n_0，度为2的节点数为n_2，则$n_0=n_2+1$。

一棵深度为k且有2^k-1个节点的二叉树称为**满二叉树**。

深度为k的，有n个节点的二叉树，当且仅当其每一个节点都与深度为k的满二叉树中编号从1至n的节点一一对应时，称之为**完全二叉树**。

性质4 具有n个节点的完全二叉树的深度为$\lfloor \log_2 n \rfloor+1$。

性质5 如果对一棵有n个节点的完全二叉树（其深度为$\lfloor \log_2 n \rfloor + 1$）的节点按层序编号（从第1层到第$\lfloor \log_2 n \rfloor + 1$层，每层从左到右），则对任一节点$i$ ($1 \leq i \leq n$)，有

（1）如果$i = 1$，则节点i是二叉树的根，无双亲；如果$i > 1$，则其双亲Parent(i)是节点$\lfloor i/2 \rfloor$。

（2）如果$2i > n$，则节点i无左孩子（节点i为叶子节点）；否则其左孩子LChild(i)是节点$2i$。

（3）如果$2i+1 > n$，则节点i无右孩子；否则其右孩子 RChild(i)是节点$2i+1$。

6.2.3 二叉树的存储结构

1. 顺序存储结构

```
//-----  二叉树的顺序存储表示  -----
typedef SqList  SqBiTree;   // 顺序二叉树指针类型为顺序表指针类型，可重用顺序表的算法代码
```

约定：将完全二叉树上编号为***i***的节点元素存储在**elem**中下标为*i*的分量中（0号单元空闲）。例如，图6.6(a)所示为图6.4(b) 完全二叉树的顺序存储结构。

对于一般二叉树，应将其每个节点与完全二叉树上的节点相对应，存储在一维数组的相应分量中，图6.4(c) 的二叉树的顺序存储结构如图6.6(b) 所示，图中以“0”表示不存在此节点。

由此可见，这种顺序存储结构仅适用于完全二叉树。因为在最坏的情况下，一个深度为***k***且只有*k*个节点的单支树（树中不存在度为**2**的节点）却需要长度为2^k-1的一维数组。

在6.4节完全二叉树的顺序存储结构将用于实现称为**堆**的抽象数据类型。

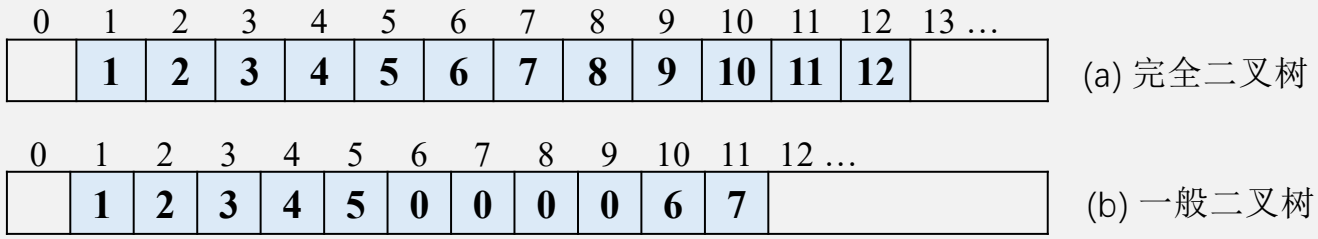
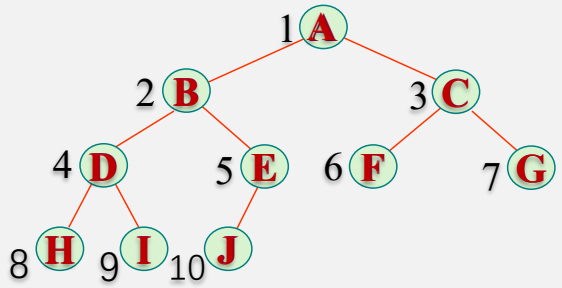


图6.6　二叉树的顺序存储结构

清華大學出版社 TSINGHUA UNIVERSITY PRESS

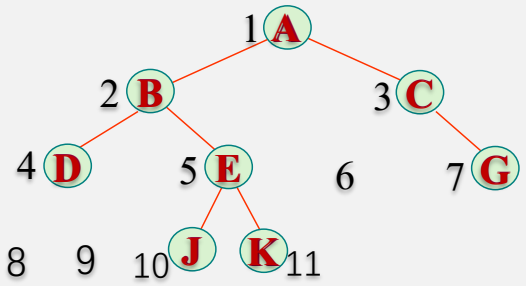
对图6.6 二叉树的顺序存储结构的解释和演示



(a) 完全二叉树 t1

	0	1	2	3	4	5	6	7	8	9	10
T1->elem											
双亲下标	0	1	1	2	2	3	3	4	4	5	
左孩子下标	2	4	6	8	10	-	-	-	-	-	
右孩子下标	3	5	7	9	-	-	-	-	-	-	

完全二叉树 t1 的顺序存储结构



(b) 非完全二叉树t2

	0	1	2	3	4	5	6	7	8	9	10	11
T1->elem												
双亲下标	0	1	1	2	2	-	3	-	-	5	5	
左孩子下标	2	4	6	8	10	-	-	-	-	-	-	
右孩子下标	3	5	7	9	11	-	-	-	-	-	-	

非完全二叉树t2的顺序存储结构

6.2.3 二叉树的存储结构（续）

2. 链式存储结构

设计不同**节点结构**可构成不同形式的**链式存储结构**。由定义可知，二叉树的节点（如图6.7(a)所示）由一个数据元素和分别指向其左、右子树的两个分支构成，表示二叉树的链表节点至少含3个域：**数据域**和**左、右指针域**，如图6.7(b)所示。为便于找到节点的双亲，还可增加一个指向其双亲节点的指针域，如图6.7(c)所示。利用这两种节点结构构成的二叉树的存储结构分别称之为**二叉链表**和**三叉链表**，如图6.8所示。

约定：把存储二叉树和树的节点的链表结点称为节点，以区分于线性结构的结点。

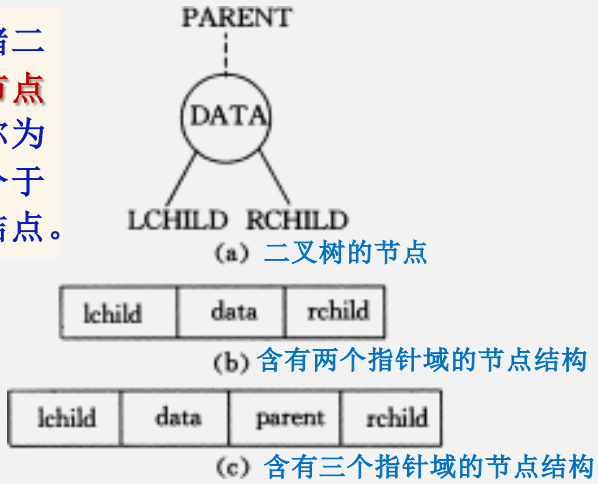


图6.7 二叉树的节点及其存储结构

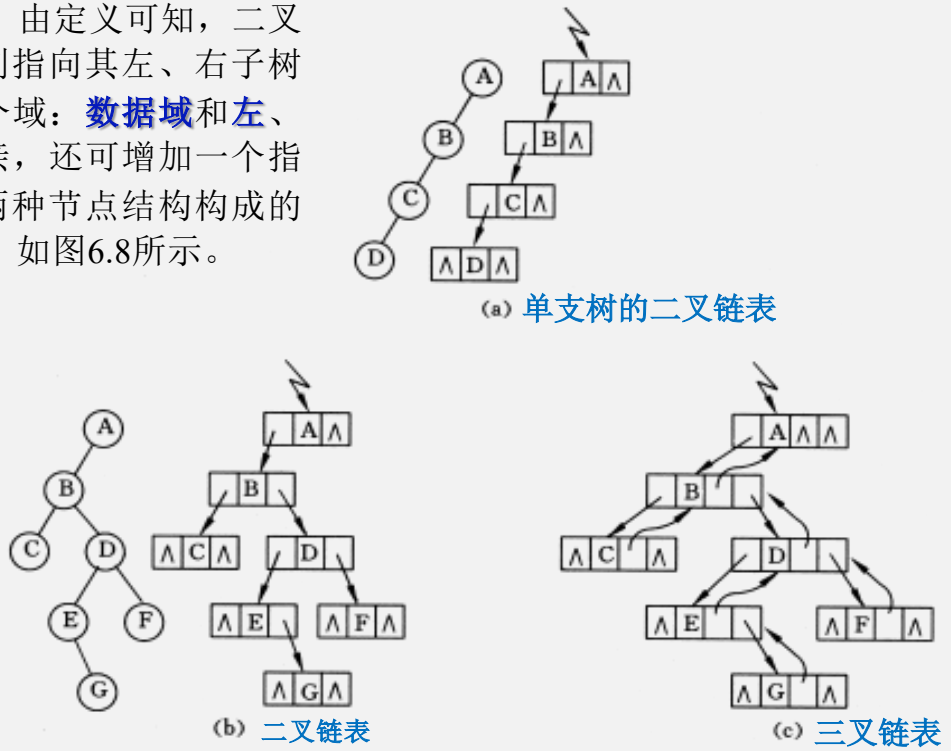


图6.8 链表存储结构

6.2.3 二叉树的存储结构（续）

```
//----- 二叉树的二叉链表存储表示 -----
typedef struct BiTNode  {
   TElemType          data;      // 数据元素
   BiTNode *lchild, *rchild;   // 左右孩子节点指针
} BiTNode, *BiTree;    // 节点类型，二叉链表指针类型
```

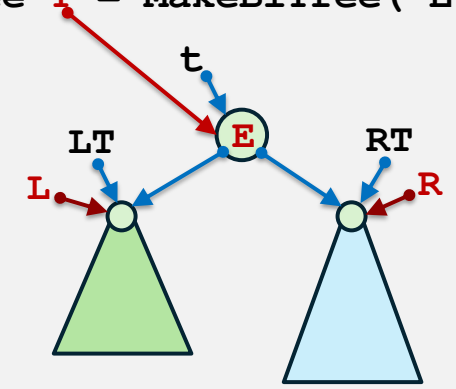
二叉链表的基本操作的实现。

（1）**组建二叉树**。顾名思义，**算法6.1先生成根节点**，和两棵已有的二叉树**组合**成一棵二叉树，并返回组建的二叉树。LT和LR可以为NULL（空二叉树），若都为空，返回的是仅含根节点的二叉树。

假设已有两棵二叉树L和R。

观察执行以下语句的过程：

```
BiTree T = MakeBiTree('E',L,R);
```




```
                                    'E'        L          R
BiTree MakeBiTree(TElemType e, BiTree LT, BiTree RT) { // 算法6.1
  // 组建并返回一棵二叉树，其中根节点的值为e，L和R分别作为左子树和右子树
  BiTree t;
  if (!(t = (BiTree)malloc(sizeof(BiTNode)))) exit(OVERFLOW);
  t->data = e;    t->lchild = LT;    t->rchild = RT;
  return t;
}
```

6.2.3 二叉树的存储结构（续）

二叉链表的基本操作的实现。

（2）**替换右子树**。用已有的二叉树**RT**替换二叉树**T**的右子树。

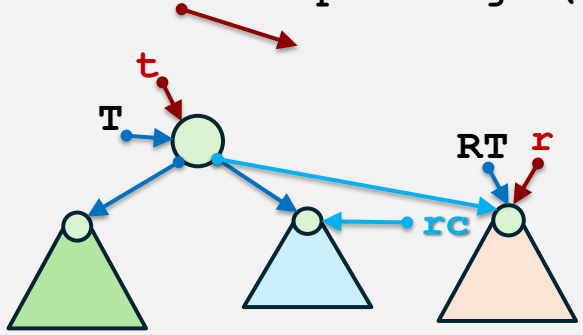
如果**T**为空则返回**NULL**，否则用**RT**替换**T**的右子树，并返回**T**的原右子树，其实现见算法6.2。

所谓替换子树，实际上具有“插入”和“删除”子树的两面性。当RT为空时，就是删除T的右子树。

假设已有两棵二叉树t和r。

观察执行以下语句的过程：

```
BiTree rt = ReplaceRight(t,r);
```




```
                                  t          r
BiTree ReplaceRight(BiTree T, BiTree RT) {  // 算法6.2 替换右子树
    // 若T非空，则用LT替换T的右子树，并返回T的原右子树
    if (NULL==T) return NULL;      // T为空
    BiTree rc = T->rchild;         // 暂存原右子树
    T->rchild = RT;                // LT作为T的右子树
    return rc;                     // 返回原右子树
}
```

6.3 遍历二叉树和线索二叉树

清華大學出版社 TSINGHUA UNIVERSITY PRESS

6.3.1 遍历二叉树

遍历二叉树（traversing binary tree）：
按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次。

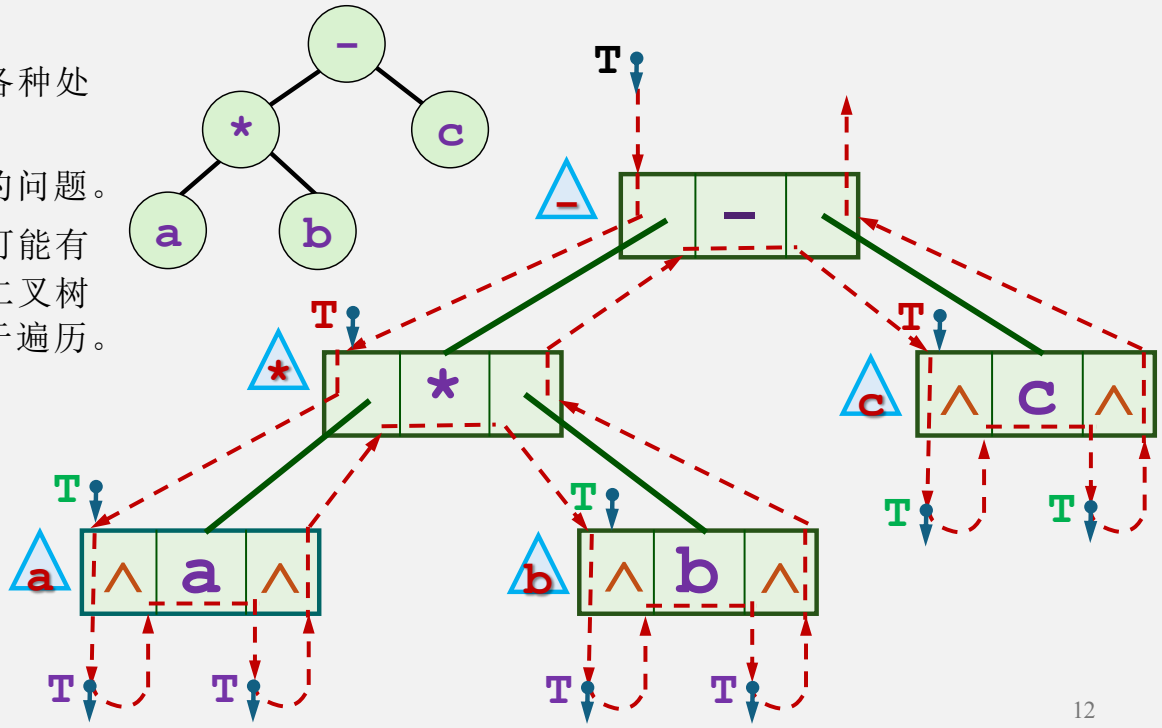
“访问”的含义很广，可以是对节点作各种处理，如输出节点的信息等。

遍历对线性结构来说，是一个容易解决的问题。

二叉树是一种非线性结构，每个节点都可能有两棵子树，因而需要寻找规律，以便使二叉树上的节点能排列在一个线性队列上，便于遍历。

```
void  Trav(BiTree T) {
  if ( T ) {
     Visit(T->data);
     Trav(T->lchild);
     Trav(T->rchild);
  }
}   // 算法6.3 先序遍历
```

对这棵二叉树依次进行先序、中序和后序遍历（所列算法是简化版本）。观察指针参数T层层递归“传递-返回”走过轨迹（算法底色表示递归层次）。

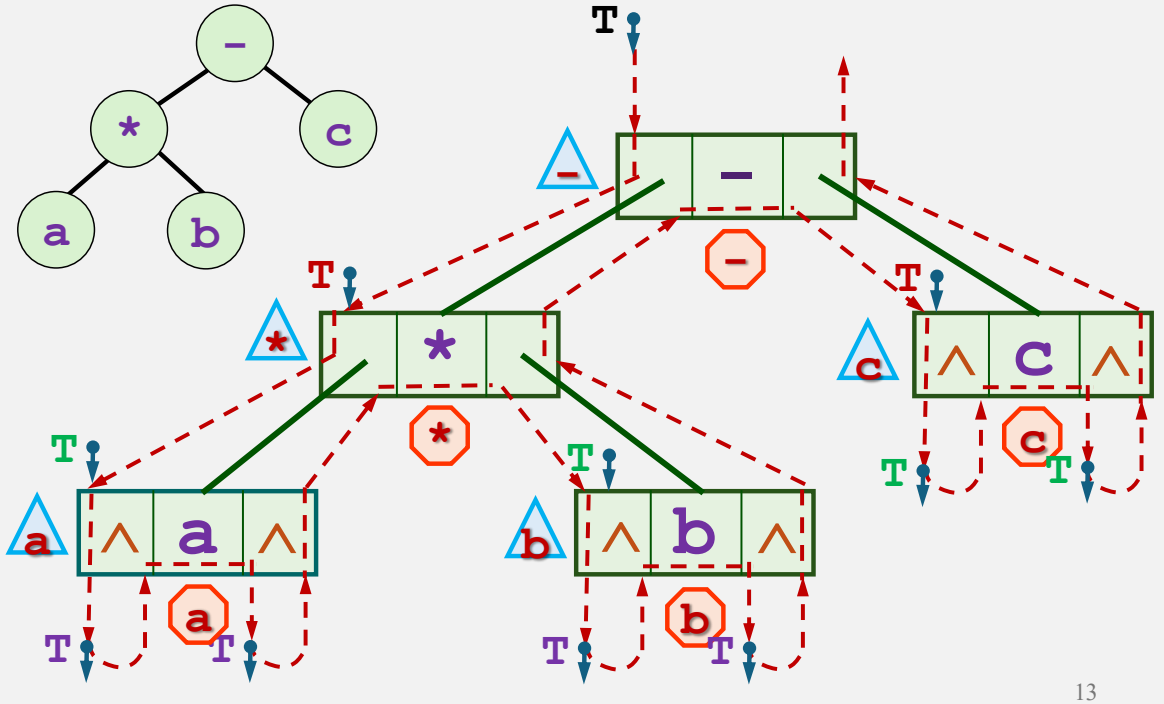


二叉树三种遍历过程演示——中序遍历

对这棵二叉树依次进行先序、中序和后序遍历（所列算法是简化版本）。
观察指针参数T层层递归“传递-返回”走过轨迹（算法底色表示递归层次）。

```
void  Trav(BiTree T) {
  if ( T ) {
     Visit(T->data);
     Trav(T->lchild);
     Trav(T->rchild);
  }
}   // 算法6.3 先序遍历
```

```
void  Trav(BiTree T) {
  if ( T ) {
     Trav(T->lchild);
     Visit(T->data);
     Trav(T->rchild);
  }
}   // 中序遍历
```



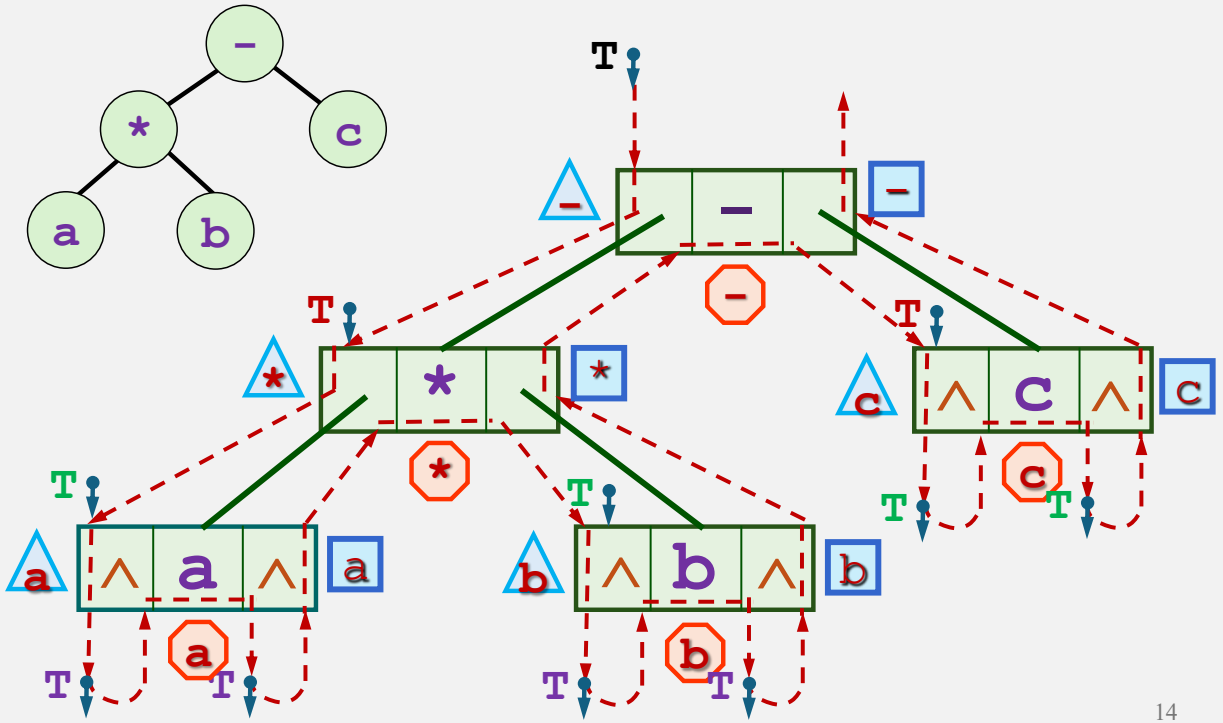
二叉树三种遍历过程演示——后序遍历

```
void  Trav(BiTree T) {
  if ( T ) {
     Visit(T->data);
     Trav(T->lchild);
     Trav(T->rchild);
  }
}   // 算法6.3 先序遍历
```

```
void  Trav(BiTree T) {
  if ( T ) {
     Trav(T->lchild);
     Visit(T->data);
     Trav(T->rchild);
  }
}   // 中序遍历算法
```

```
void  Trav(BiTree T) {
  if ( T ) {
     Trav(T->lchild);
     Trav(T->rchild);
     Visit(T->data);
  }
}   // 后序遍历算法
```

对这棵二叉树依次进行先序、中序和后序遍历（所列算法是简化版本）。
观察指针参数T层层递归“传递-返回”走过轨迹（算法底色表示递归层次）。

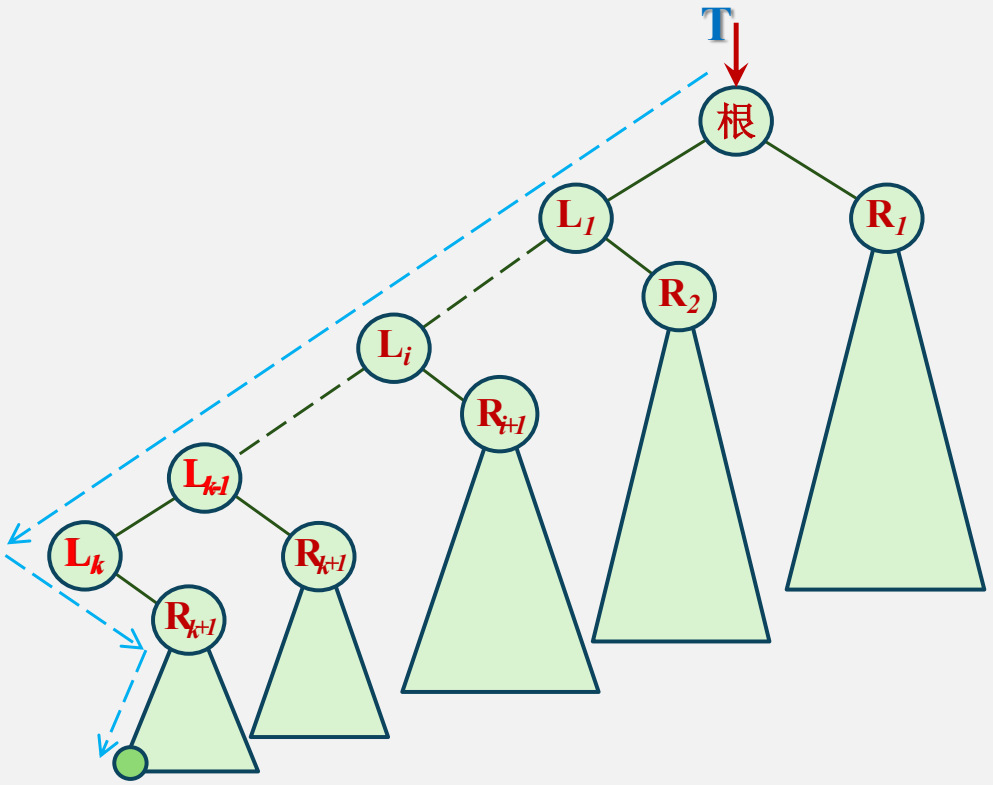


算法6.4 使用栈的中序非递归遍历（1）

清華大學出版社 TSINGHUA UNIVERSITY PRESS

二叉树从根结点出发的“左路径”是唯一的，如右图所示。

- **中序遍历访问的第一个结点?**
  - 根结点出发的左路径的末梢结点，也就是“最左下”(其左子树为空)的结点L_k
- **中序遍历访问的下一个结点?**
  - 若右子树不空，为对其以R_{k+1}为根的右子树进行中序遍历时访问的第一个结点，也就是该子树的“最左下”结点；
  - 若右子树为空，或者遍历R_{k+1}子树后返回，则访问的下一个结点为L_{k-1}。
- **如何找到L_{k-1}?**
  - 利用栈



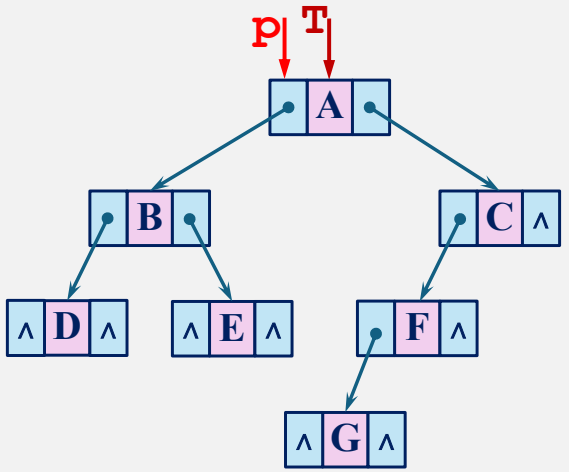
算法6.4 使用栈的中序非递归遍历（1）

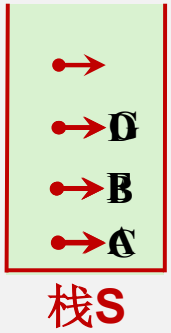
清華大學出版社 TSINGHUA UNIVERSITY PRESS

遍历序列： D B E A G F C

算法6.4 是本书讨论的使用栈辅助的中序非递归遍历的两个算法之一。

```
Status InOrderTraverse(BiTree T, Status (*Visit)(TElemType)) {
  // 算法6.4。采用二叉链表存储结构，Visit是对数据元素操作的应用函数。
  // 中序遍历二叉树T的非递归算法，对每个数据元素调用函数Visit。
  BiTree p;
  Stack S = InitStack(10, 5);  Push(S, T);  // 根指针进栈
  while (!StackEmpty(S)) {     // 遍历到栈空为止
    while (p = GetTop(S))
      Push(S, p->lchild);      // 向左走到尽头
    p = Pop(S);                // 空指针退栈
    if (!StackEmpty(S)) {      // 访问结点，向右一步
      p = Pop(S);              // 栈顶的指针退栈到p
      if (!Visit(p->data)) return ERROR;   // 访问P结点
      Push(S, p->rchild);      // P结点右指针入栈
    }
  }
  return OK;
}
```





算法6.5 使用栈的中序非递归遍历（2）

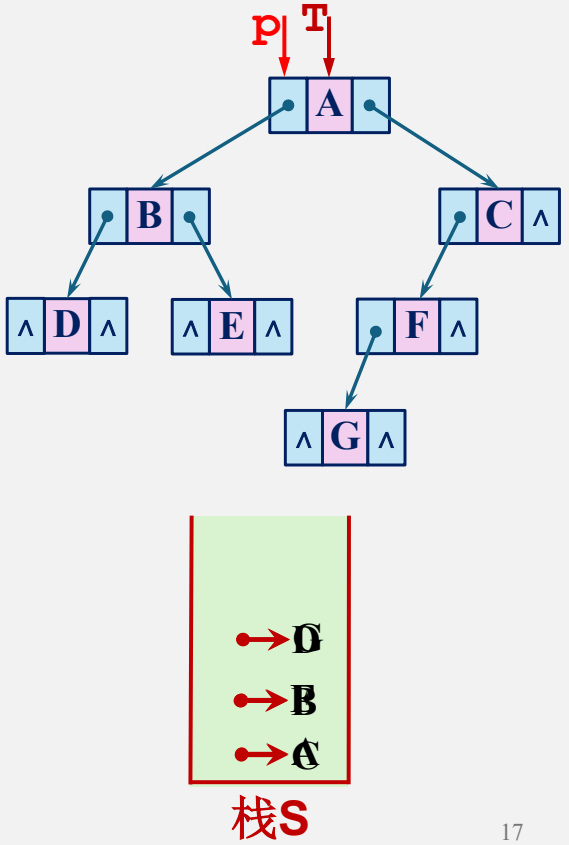
清華大學出版社 TSINGHUA UNIVERSITY PRESS

遍历序列：D B E A G F C

算法6.5 是本书讨论的使用栈辅助的中序非递归遍历的第二个算法。

将算法6.4的二重循环简约为一重循环，且空指针不进栈，大幅减少栈操作。

```
Status InOrderTraverse(BiTree T, Status (*Visit)(TElemType)) {
  // 算法6.5。采用二叉链表存储结构，Visit是对数据元素操作的应用函数。
  // 中序遍历二叉树T的非递归算法，对每个数据元素调用函数Visit。
  BiTree p;
  Stack S = InitStack(10,5);  p = T;    // 栈初始化，p指向根结点
  while (p || !StackEmpty(S)) { // 遍历到p和栈均为空为止
    if (p){ Push(S, p);  p = p->lchild; } // p不空，进栈，向左走
    else {  // 根指针退栈，访问结点，遍历右子树
      p = Pop(S);   if (!Visit(p->data)) return ERROR;
      p = p->rchild; // 向右一步
    }
  }
  return OK;
}
```



算法6.7 先序递归构造二叉树的二叉链表

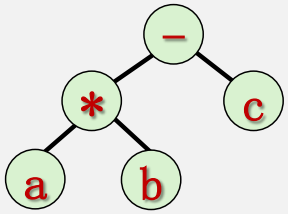
清華大學出版社 TSINGHUA UNIVERSITY PRESS

算法6.7 基于一棵二叉树的先序序列，构造其二叉链表存储结构。

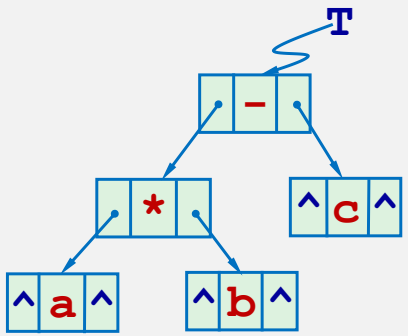
例如，图6.10的二叉树及其先序序列。

在先序序列增添的#号(教材用空格)，与结点的空指针一一对应。

请看它与二叉链表构造的对应关系。



先序序列： -*a##b##c##




```
BiTree CreateBiTree(char *pts) {  // 算法6.7
   // 由先序串pts构造并返回二叉树的二叉链表，全局变量i_为pts当前字符的位标，初值为0
   BiTree T;
   if (pts[i_] == '#') T = NULL; // 空树('#'代表空格，更直观)
   else {
      if(!(T = (BiTNode*)malloc(sizeof(BiTNode)))) exit(OVERFLOW); //分配结点空间
      T->data = pts[i_++] ;                    // 生成根结点
      T->lchild = CreateBiTree(pts);   i_++;   // 构造左子树
      T->rchild = CreateBiTree(pts);           // 构造右子树
   }
   return T;
}
```

算法6.7 先序递归构造二叉链表

示例，设有外部声明：

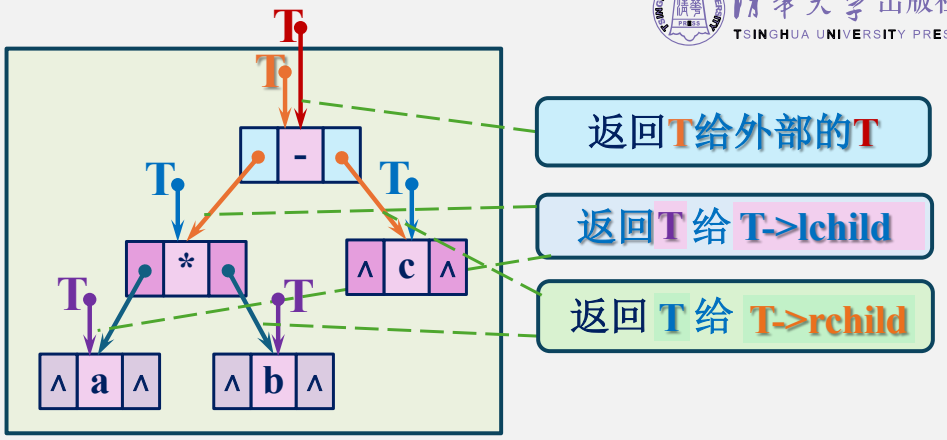
```
char  *s="-*a##b##c##";
int    i = 0;
BiTree  T;
```

串 s 是图6.10的二叉树的先序序列。

调用算法6.7：

```
i_ = 0;
T = CreateBiTree(s);
```

可构造二叉树T的二叉链表存储结构。

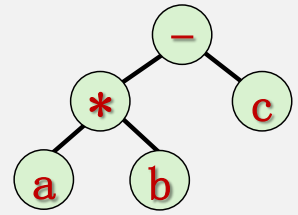


返回T到外部非递归调用层的T。

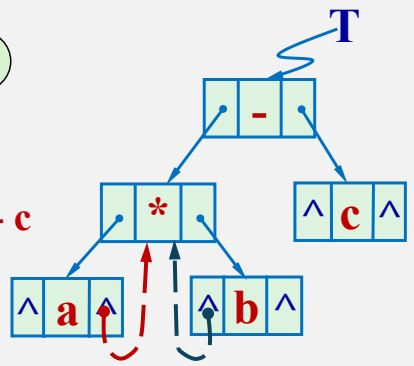
```
BiTree CreateBiTree(char *pts) {  // 算法6.7
   // 由先序串pts构造并返回二叉树的二叉链表，全局变量i_为pts当前字符的位标，初值为0
   BiTree T;
   if (pts[i_] == '#') T = NULL; // 空树('#'代表空格，更直观)
   else {
      if(!(T = (BiTNode*)malloc(sizeof(BiTNode)))) exit(OVERFLOW); //分配结点空间
       T->data = pts[i_++] ;                        // 生成根结点
      T->lchild = CreateBiTree(pts);    i_++;      // 构造左子树
      T->rchild = CreateBiTree(pts);               // 构造右子树
   }
   return T;
}
```

6.3.2 线索二叉树

算法6.8 中序遍历二叉线索树



中序序列：a * b - c



何谓二叉线索树？

例如，图6.10的二叉树、二叉链表及中序序列。

中序**线索化**，就是将二叉链表结点的空指针域，改为中序**线索**：

左链为**向前线索**，指向在中序序列中的前驱；（例如结点b的左链）

右链为**向后线索**，指向在中序序列中的后继。（例如结点a的右链）

为了区分结点左、右链域是指针还是线索，在结点增设左、右标记域，用0和1予以区分。

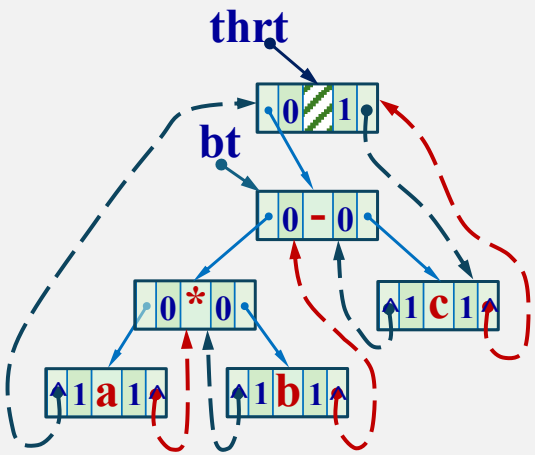
增设了一个**头结点**，左链指向根结点，右链为线索，指向最后访问的结点。头结点可视为最先且最后访问的结点。

请看二叉链表的中序线索化概要演示。

（详细讲演请看**算法6.9**和**6.10**）

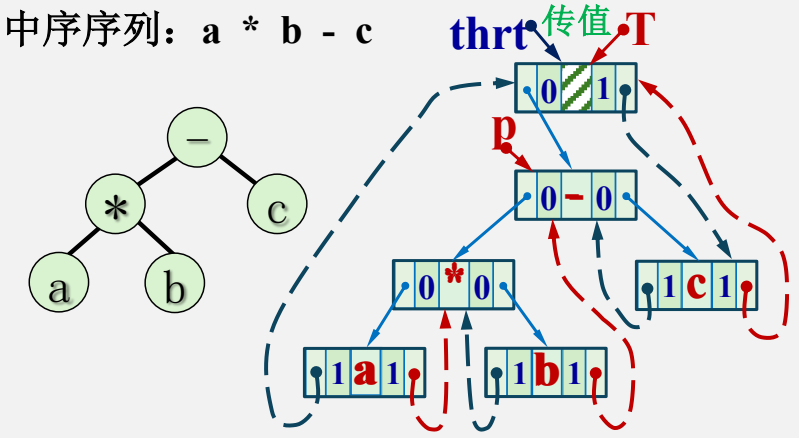
二叉链表的中序线索化，支持对其进行无栈中序遍历，以及各种基于中序遍历框架的应用算法开发。

还可以进行先序或后序线索化，分别支持先序或后序遍历及应用。



算法6.8 中序遍历二叉线索树

算法6.8 基于一棵二叉树的中序线索化二叉链表，进行中序遍历。
可和用栈辅助的非递归中序遍历算法6.4和6.5对比，算法6.8利用中序后继线索（棕红色虚线箭头），实现了无栈非递归中序遍历。



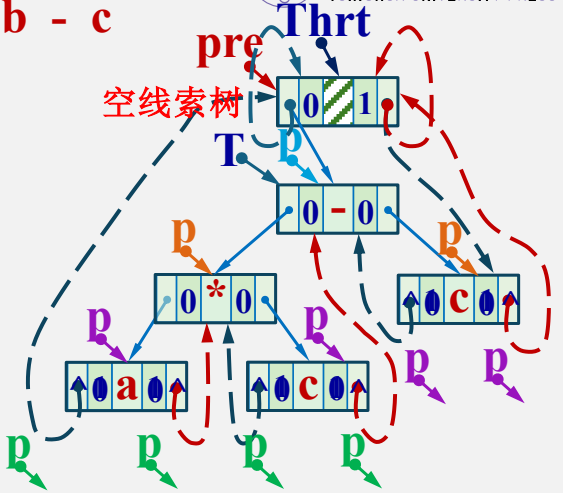

```
Status InOrderTrav_Thr(BiThrTree T, Status (*Visit)(TElemType)) {   // 算法6.8
   // T指向头结点，头结点的左链lchild指向根结点，头结点的右链lchild指向中序遍历的最后一个结点。
   // 中序遍历二叉线索链表表示的二叉树T，对每个数据元素调用函数Visit。
   BiThrTree p = T->lchild;   // p指向根结点
   while (p != T) {   // 空树或遍历结束时，p回到了头结点，此时p==T
      while (p->LTag==Link/*0*/) p = p->lchild;   // 顺着左指针链走到 “左下角”
      if (!Visit(p->data)) return ERROR;          // 访问其左子树为空的结点，若访问有错则报错
      while (p->RTag==Thread && p->rchild!=T)
         { p = p->rchild;  Visit(p->data); }      // 经右线索到达中序后继结点，并访问之
      p = p->rchild;   // 经右指针，p进至其右子树根
   }
   return OK;   结束，返回
}
```

算法6.9和6.10 二叉树中序线索化

中序序列：头 a * b - c

```
BiThrTree InOrderThreading(BiThrTree T) { // 算法6.9
  // 中序遍历二叉树T，并将其中序线索化，Thrt指向头结点。
  BiThrTree Thrt;
  if(!(Thrt=(BiThrTree)malloc(sizeof(BiThrNode))))exit(OVERFLOW);
  Thrt->LTag = Link;  Thrt->RTag =Thread;    // 建头结点
  Thrt->rchild = Thrt;                       // 右线索回指
  if(!T) Thrt->lchild = Thrt;                // 若二叉树空，则左指针回指
  else {
    Thrt->lchild = T;   pre = Thrt;  // pre将作为前驱指针辅助线索化
    InThreading(T);     // 调用算法6.10，递归中序遍历 T 进行中序线索化
    pre->rchild = Thrt;  pre->RTag = Thread; // 最后一个结点线索化
    Thrt->rchild = pre;
  }
  return Thrt;
}
```

```
void InThreading(BiThrTree p) { // 递归中序线索化  算法6.10
  if ( p ) {      // 在递归中序遍历过程中，外部指针pre和值参p总是互为前驱和后继指针
    InThreading(p->lchild);    // 左子树线索化
    if(!p->lchild) { p->LTag = Thread;  p->lchild = pre; }      // 建前驱线索
    if(!pre->rchild) { pre->RTag = Thread;  pre->rchild = p; } // 建后继线索
    pre = p;                   // 保持pre指向p的前驱
    InThreading(p->rchild);    // 右子树线索化
  }
}
```



关注：
引用形参的作用和pre在递归遍历中的轨迹，以及值参p的特点。

6.4 堆和优先队列

6.4.1 优先队列

按**重要性**或**优先级**来组织的队列称为**优先队列**（priority queue）。在第3章讨论的**普通队列**中，每次出队的总是队头元素。如果要改为出队的是最高优先级元素，则有两种简单的改进方法：

1. 不改变入队操作（总是在队尾入队）。出队操作就要在队列中**查找最高优先级元素**，时间代价是$O(n)$的（n为队列长度）。删除该出队元素，顺序循环队列的时间代价是$O(n)$的，而链队列是$O(1)$的。总的时间代价是$O(n)$的。

2. 改变入队操作。入队的元素**按优先级保序插入**队列中，使得队列维持按优先级有序，但时间代价是$O(n)$的。这样，出队的时间代价是$O(1)$的，但综合代价仍是$O(n)$的。

优先队列应用广泛。很多编程语言的标准库也提供了优先队列数据类型，比如C++的priority_queue、Java的PriorityQueue。

在后续章节中，**赫夫曼树构造算法、求最小生成树的克鲁斯卡尔算法、求最短路径的迪杰斯特拉算法、堆排序算法、外部排序的多路平衡归并算法**等，都可用优先队列辅助提高时间性能。因此，优先队列需要选择操作效率更高的存储表示和实现的方式。

为处理便利，通常把优先级量化为整数或实数。但在应用场景，可能越小越优先，也可能越大越优先。可称**最高优先级**为**Top**，而不是Min或Max。优先队列的**入队**和**出队**操作接口可表述为：

```
Status Insert(PriorityQueue PQ, PQElemType e);
   // 若优先队列PQ已满，则扩容；按元素e的优先级插入到优先队列PQ合适位置，并返回OK;
PQElemType DelTop(PriorityQueue PQ);
   // 若优先队列PQ不空，则删除并返回最高优先级的元素，否则返回nullE（空元素）
```

6.4.2 堆的定义

堆是一类**完全二叉树**，可用于高效地实现排序、选择最小（大）值和优先队列等。

堆是具有以下特性的**完全二叉树**，其所有非叶子节点均不大于（或不小于）其左右孩子节点，即按完全二叉树的节点编号排列，n个节点的关键字序列（k_1, k_2, …, k_n）称为**堆**，当且仅当满足以下关系：

小顶堆：$ki \le k2i$　$ki \le k2i+1$

或　**大顶堆**：$ki \ge k2i$　$ki \ge k2i+1$　$(i = 1, 2, \ldots, \lfloor n/2 \rfloor)$

其中，若堆中所有非叶子节点均不大于其左右孩子节点，则称为**小顶堆**（**小根堆，最小堆**），图6.13(a)给出了一个小顶堆示例；

若堆中所有非叶子节点均不小于其左右孩子节点，则称为**大顶堆**（**大根堆，最大堆**）。

堆中的子树称为**子堆**。图6.13(b)给出了一个大顶堆示例。图6.13(c)给出的完全二叉树中，根节点58既不满足小顶堆特性，也不满足大顶堆特性，因此不是堆。

堆中**根节点的位置**称为**堆顶**，**最后节点的位置**称为**堆尾**，**节点个数**称为**堆长度**。

由定义可知，小顶堆的堆顶节点必定为n个节点的最小值，而大顶堆的堆顶节点必定为n个节点的最大值。为了统一讨论大顶堆和小顶堆的实现，**引入节点的优先级的概念，对于大顶堆，关键字大者优先；对于小顶堆，关键字小者优先。**

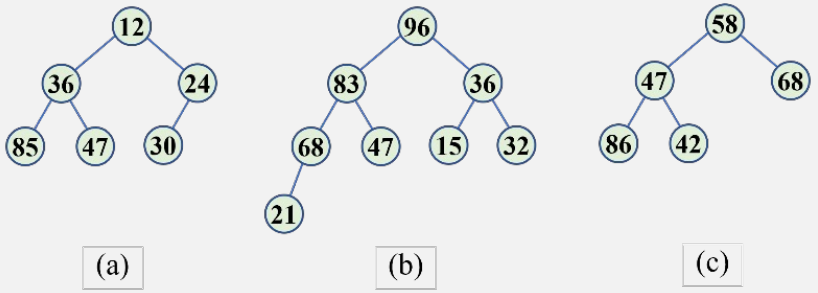


图6.13 堆的示例和反例
(a) 小顶堆； (b) 大顶堆； (c) 不是堆

清華大學出版社
TSINGHUA UNIVERSITY PRESS



堆的存储表示

堆是一棵完全二叉树。之前已将完全二叉树定义为顺序表指针类型。

类似地，定义堆为顺序表类型（小顶堆和大顶堆的主要差别在与操作中调用的优先级比较函数不同），并将数据元素的类型定义为含有两个域的结构体，分别是用于比较优先级的关键字域**key**，与应用相关的信息域**other**（在算法中被省略）。

```
typedef struct {
    KeyType     key;        // 关键字域
    OtherType   other;      // 其他数据域（除关键字之外的其他数据信息）
} ElemType;                 // 数据元素类型（这里是含关键字的多域数据类型，可按应用需求定义）
typedef SqList  Heap;       // 与顺序表相同的堆指针类型
```

数据元素的关键字项可根据实际情况确定，但关键字类型应是可比较的。不同的元素类型，要定义相应的优先函数。假设可使用比较运算，**大顶堆**和**小顶堆**的**优先函数**可分别定义为：

```
int greatPrior(ElemType x, ElemType y) { return x.key>=y.key; } // 大顶堆优先函数
int lessPrior(ElemType x, ElemType y) { return x.key<=y.key; }  // 小顶堆优先函数

int Prior(ElemType x, ElemType y);    // 堆操作的优先函数变量，建堆时赋予相应的优先函数
```

在实际应用中，堆的用途广泛，对不同的元素类型，需要定义对应的优先函数，并赋予优先函数变量Prior。在第7章有多个算法用到了堆，将可看到不同的优先函数。

6.4.3 堆的基本操作的实现

1. 堆的筛选——算法6.11

堆的**筛选操作**是将一个可能尚未成堆的二叉树中指定的以pos节点为根的子树调整为子堆，其前提是pos节点的左右子树均为子堆。**筛选操作的过程**为：将pos节点与左右孩子中较优先者比较，若pos节点较优先则结束；否则pos节点与较优先的孩子交换位置，pos位标下移。重复上述步骤，直到pos指示叶子节点为止。

例如，把以pos节点为根的子树（49，76，63，52，49）筛选为大顶堆，其中节点49有两个，为以示区别，第一个49带下划线。

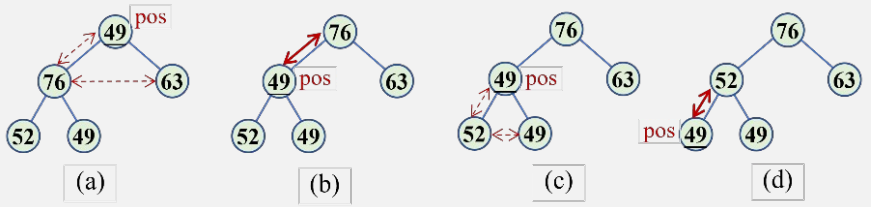
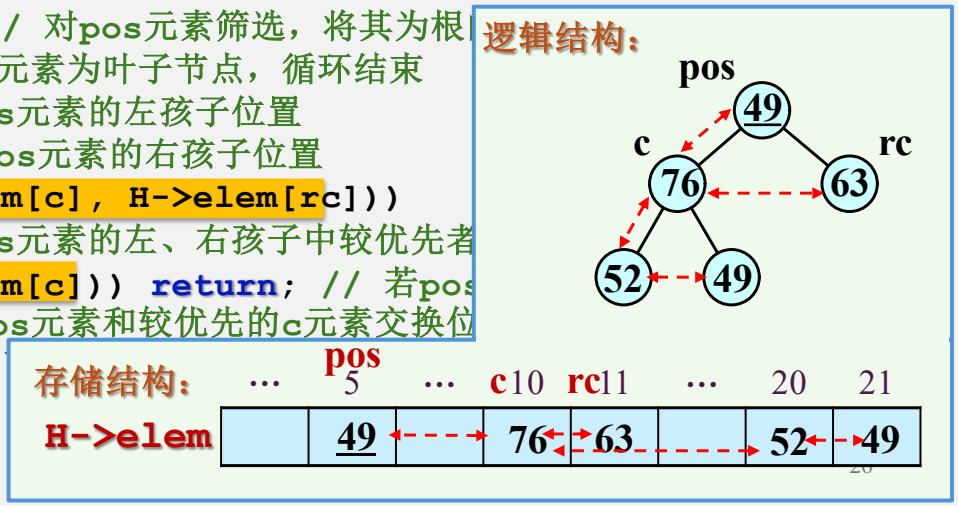


图6.14　调整过程示意图

```
void SiftDown(MaxHeap H, int pos) { // 对pos元素筛选，将其为根
  while (pos <= H->len/2) {  // 若pos元素为叶子节点，循环结束
    int c = pos*2;           // c为pos元素的左孩子位置
    int rc = pos*2+1;        // rc为pos元素的右孩子位置
    if (rc <= H->len && !Prior(H->elem[c], H->elem[rc]))
      c = rc;                // c为pos元素的左、右孩子中较优先者
    if (H->prior(H->elem[pos], H->elem[c])) return; // 若pos
    swapElem(H, pos, c);     // 否则pos元素和较优先的c元素交换位
    pos = c;                 // 继续向
  }
}
```



6.4.3 堆的基本操作的实现（续）

2. 堆的插入——算法6.12（向上调整）

堆的**插入操作**是**将插入元素加到堆尾**，此时需判别堆尾和其双亲节点是否满足堆特性，若不满足，则需要进行**向上调整**，将插入元素与双亲交换；交换后，插入元素若存在双亲且此双亲节点不满足堆特性，则需要重复上述过程，因此插入堆的**操作步骤**如下：

（1）将插入元素加到堆尾，并用curr指示堆尾；

（2）若curr指示堆顶，插入操作结束，否则，将curr节点与其双亲节点比较，若curr节点较优先则交换，curr上移，重复（2）；否则插入操作结束。

例如，在堆（76, 52, 63, 49, 49）中插入81。过程如图6.15所示。

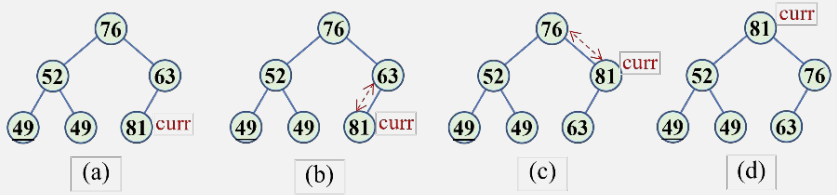
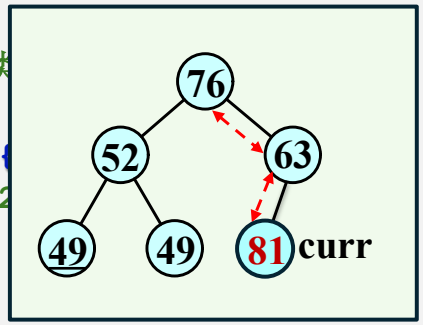


图6.15 插入过程示意图
(a)插入81到堆尾； (b)81与63比较；
(c)81与76比较； (d)81与76交换位置

```
Status Insert(MaxHeap H, ElemType e) {  // 将元素e插入至堆H中
    if (H->len >= H->size) reallocHeap(H);      // 堆已满，则扩容（类
    int curr = ++H->len;   H->elem[curr] = e;   // 将元素e加到堆尾
    while (1 != curr && !Prior(H->elem[curr/2], H->elem[curr])) {
        swapElem(H, curr, curr/2);              // 交换curr与curr/2
        curr /= 2;
    }
    return OK;
}
```



插入操作是从叶子节点**向上调整**的过程，和筛选操作方向相反，最坏情况下，比较次数为堆的高度减1，因此堆插入操作的算法时间复杂度为$O(\log n)$。

6.4.3 堆的基本操作的实现（续）

3. 删除堆顶节点——算法6.13

删除堆顶节点时，用堆尾节点代替堆顶节点，不影响其左右子堆的特性，但需要对新的堆顶节点进行筛选，以维护整个堆的特性。删除堆顶节点的操作步骤为：

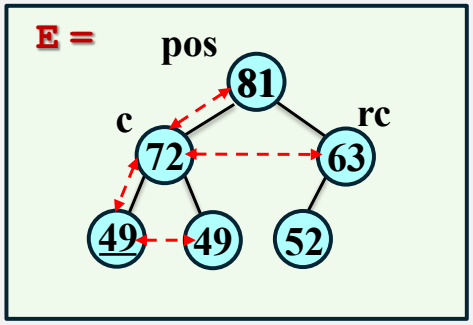
（1）取出堆顶节点；

（2）将堆顶节点与堆尾节点交换位置，并将堆长度减1；

（3）对堆顶节点进行筛选。

算法6.13实现了删除堆顶元素的操作。基本操作SeeTop只需返回堆顶元素，

删除

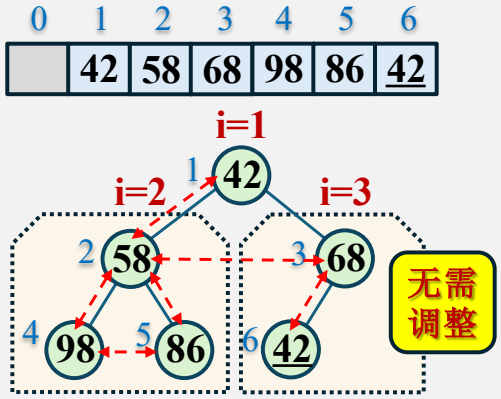



```
Ele
}
```

```
void SiftDown(MaxHeap H, int pos) { // 对pos元素筛选，将其为根的子树调整为子堆
  while (pos <= H->len/2) {  // 若pos元素为叶子节点，循环结束
    int c = pos*2;           // c为pos元素的左孩子位置
    int rc = pos*2+1;        // rc为pos元素的右孩子位置
    if (rc <= H->len && !Prior(H->elem[c], H->elem[rc]))
      c = rc;                // c为pos元素的左、右孩子中较优先者的位置
    if (H->prior(H->elem[pos], H->elem[c])) return; // 若pos元素较优先，则结束
    swapElem(H, pos, c);     // 否则pos元素和较优先的c元素交换位置
    pos = c;                 // 继续向下调整
  }
}
```

清華大學出版社 TSINGHUA UNIVERSITY PRESS

4. 建堆——算法6.14



由于单个节点的完全二叉树满足堆特性，所以叶子节点都是堆。对n个节点的完全二叉树建堆的过程是，依次将以编号$n/2$，$n/2-1$，…，1的节点为根的子树筛选为子堆。

```
MaxHeap MakeHeap(SqList L) { // 返回由顺序表L构建的堆
  for(int i=L->len/2; i>0; i--)
    SiftDown(L, i);  // 对以i节点为根的子树进行筛选
  return L;          // 返回构建的堆
}
```

例如，对初始序列（42, 58, 68, 98, 86, 42）建大顶堆的过程如图6.16所示。由于堆长度为6，所以只需依次对编号为3，2，1的节点筛选。

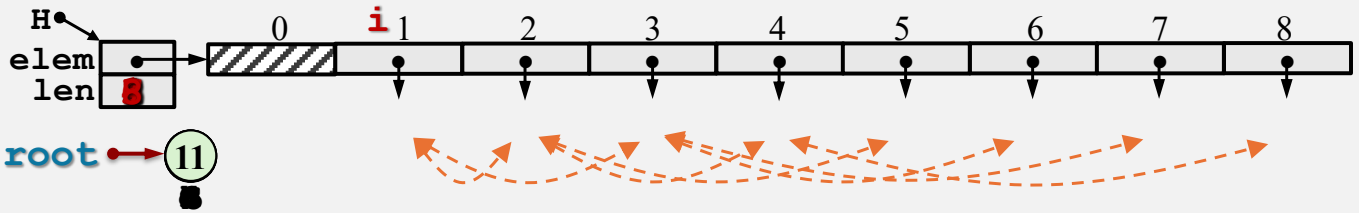
深度为h的堆中第i层上的节点数最多2^{i-1}，以它们为根的二叉树的深度为$h-i+1$，筛选算法中进行的关键字比较次数为$2(h-i)$，则建堆总共进行的比较次数为：

$$\sum_{i=h-1}^{1} 2^{i-1}\times 2(h-i) = \sum_{i=h-1}^{1} 2^{i}\times (h-i) = \sum_{j=1}^{h-1} 2^{h-1}\times j \le 2n\sum_{j=1}^{h-1} j/2^{j} \le 4n$$

因此，建立初始堆总共进行比较次数不会超过$4n$，其时间复杂度为$O(n)$。

(a) 初始序列；(b) 3个节点筛选结果；(c) 2个节点筛选结果；(d) 1个节点筛选结果

图6.16 建立初始堆

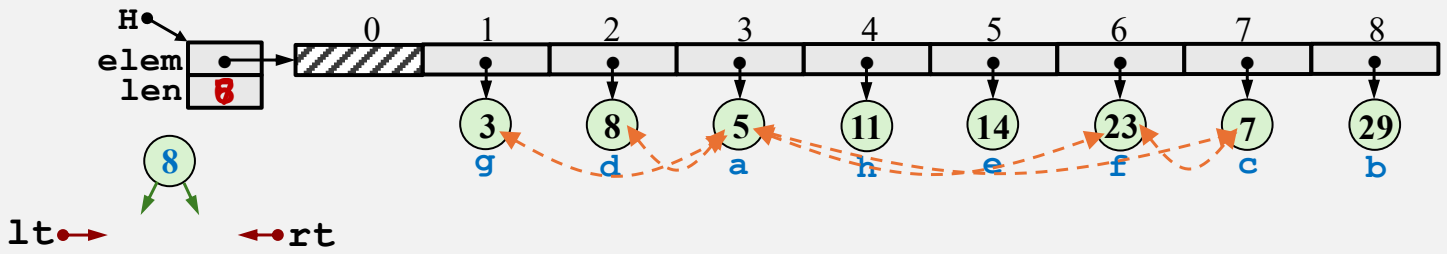


例6-1 第一步：构建赫夫曼树——算法6.15

```
char ds[8]="abcdefgh";                  // 8个字符
int  ws[8]={5,29,7,8,14,23,3,11};       // 对应的8个权值
int n=8;
Prior=CompareNode;  // 树的根节点（权值）之间比较函数作为小根堆的优先判定函数
HT = MakeHFTree(ds, ws, n);  // 用ds和ws构造并返回赫夫曼树根指针
```

```
HFTNode *MakeHFTree(char data[], int weight[], int n) { //算法6.15 构造赫夫曼树
  // 由长度为n的字符数组data和权值数组weight构造并返回赫夫曼树，使用小顶堆存储森林
  HFTNode *root, *lt, *rt;
  Heap H = InitHeap(n+1);              // 新建容量为n+1的空堆
  for (int i=0; i<n; i++) {            // 生成每个权值节点并插入小顶堆H
    if (!(root=(HFTNode*)malloc(sizeof(HFTNode)))) exit(OVERFLOW); // 根节点
    root->data = data[i];              // 字符
    root->weight = weight[i];          // 权值
    Insert(H, root);                   // 插入小顶堆
  }                                    // for循环结束时，优先队列内是单节点树的森林
  ... ...   /* 构造赫夫曼树 */
  return root;                         // 返回构造的赫夫曼树
}
```

建堆

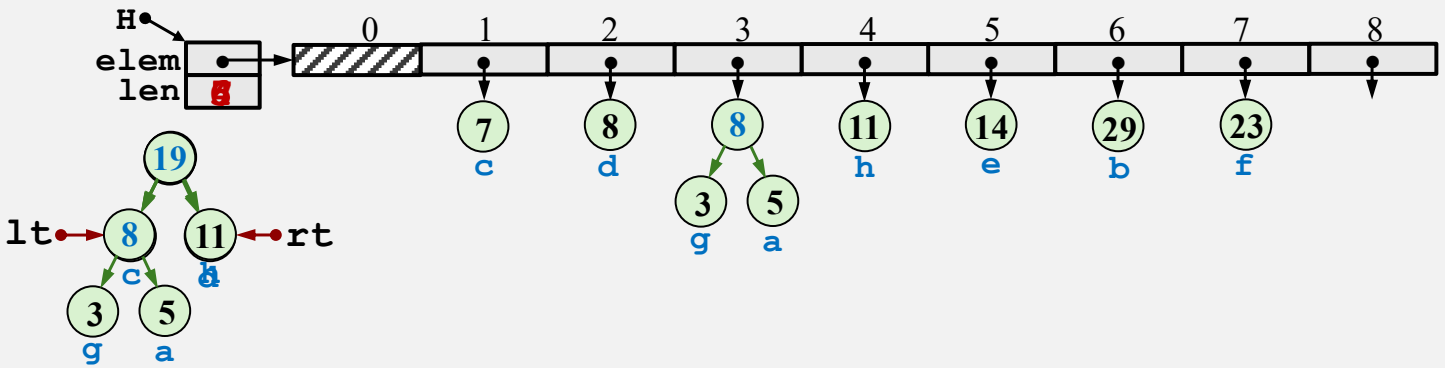


堆的两个操作要点：

- DelTop(H) 可能发生一次向下调整过程；
- Insert(H, root) 可能发生一次向上调整过程。

建赫夫曼树

```
HFTNode *MakeHFTree(char data[], int weight[], int n) { //算法6.15 构造赫夫曼树
  // 由长度为n的字符数组data和权值数组weight构造并返回赫夫曼树，使用小顶堆存储森林
  ... ...  /* 建小顶堆H */
  while(H->len > 1) {                    // while循环每次将权值最小的两棵树合二为一
    lt = DelTop(H);   rt = DelTop(H);          // 从堆取权值最小两棵树为左、右子树
    if (!(root=(HFTNode*)malloc(sizeof(HFTNode)))) exit(OVERFLOW); // 根节点
    root->data = '*';                                  // 内部节点数据域为'*'
    root->weight = lt->weight + rt->weight;    // 根节点的权值为左右孩子权值之和
    root->lchild = lt;   root->rchild = rt;    // 接上左右子树
    Insert(H, root);                                   // 将组合成的新树插入堆
  }  // 每次合二为一减少了一棵树，while循环结束时，优先队列只含的一棵树即为所求
  return root;                        // 返回构造的赫夫曼树
}
```



清華大學出版社
TSINGHUA UNIVERSITY PRESS

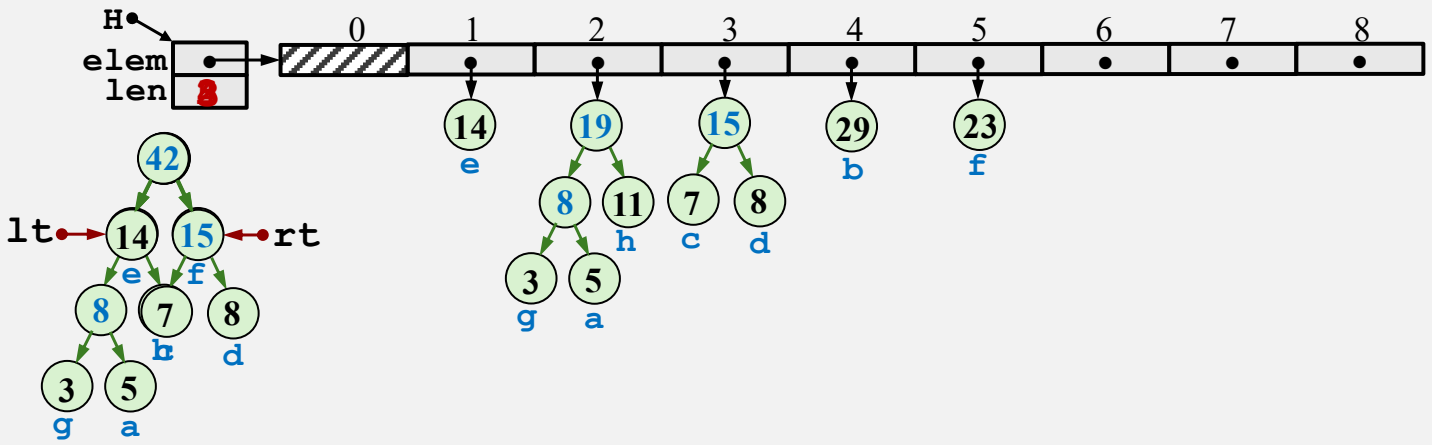
堆的两个操作要点：

- DelTop(H) 可能发生一次向下调整过程；
- Insert(H, root) 可能发生一次向上调整过程。

以下简化演示过程

建赫夫曼树

```
HFTNode *MakeHFTree(char data[], int weight[], int n) { //算法6.15 构造赫夫曼树
  // 由长度为n的字符数组data和权值数组weight构造并返回赫夫曼树，使用小顶堆存储森林
  ... ...  /* 建小顶堆H */
  while(H->len > 1) {                    // while循环每次将权值最小的两棵树合二为一
    lt = DelTop(H);   rt = DelTop(H);          // 从堆取权值最小两棵树为左、右子树
    if (!(root=(HFTNode*)malloc(sizeof(HFTNode)))) exit(OVERFLOW); // 根节点
    root->data = '*';                                  // 内部节点数据域为'*'
    root->weight = lt->weight + rt->weight;    // 根节点的权值为左右孩子权值之和
    root->lchild = lt;   root->rchild = rt;    // 接上左右子树
    Insert(H, root);                                   // 将组合成的新树插入堆
  }  // 每次合二为一减少了一棵树，while循环结束时，优先队列只含的一棵树即为所求
  return root;                        // 返回构造的赫夫曼树
}
```



清華大學出版社 TSINGHUA UNIVERSITY PRESS

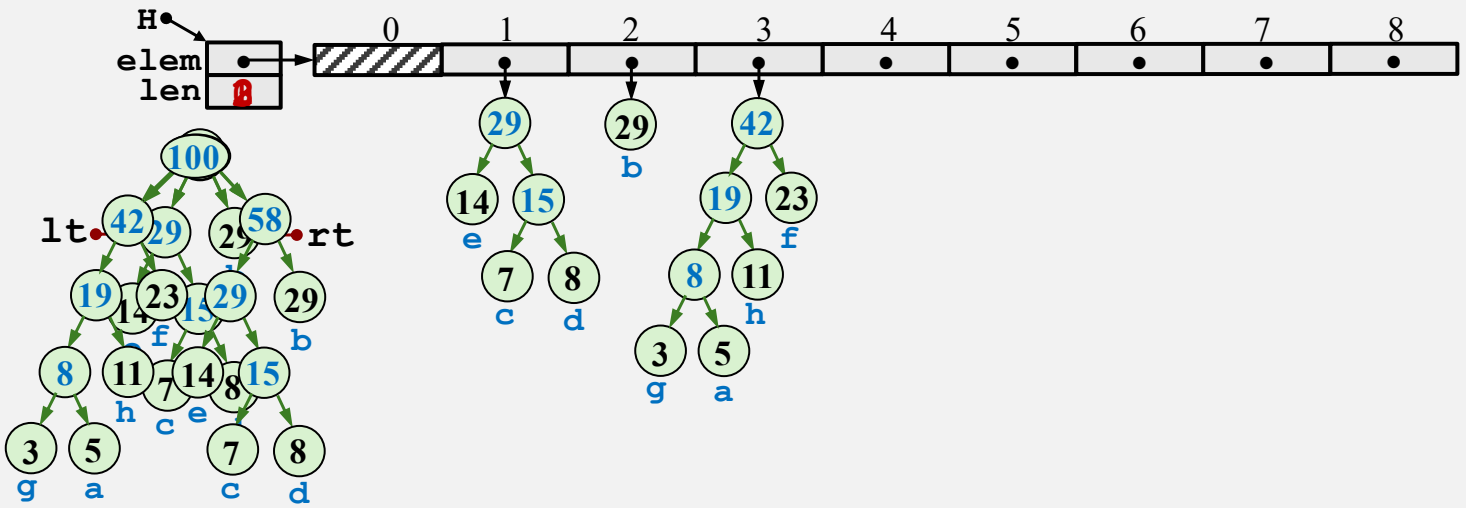
堆的两个操作要点：

- DelTop(H) 可能发生一次向下调整过程；
- Insert(H, root) 可能发生一次向上调整过程。

以下简化演示过程

建赫夫曼树

```
HFTNode *MakeHFTree(char data[], int weight[], int n) { //算法6.15 构造赫夫曼树
  // 由长度为n的字符数组data和权值数组weight构造并返回赫夫曼树，使用小顶堆存储森林
  ... ...  /* 建小顶堆H */
  while(H->len > 1) {                    // while循环每次将权值最小的两棵树合二为一
    lt = DelTop(H);   rt = DelTop(H);          // 从堆取权值最小两棵树为左、右子树
    if (!(root=(HFTNode*)malloc(sizeof(HFTNode)))) exit(OVERFLOW); // 根节点
    root->data = '*';                                  // 内部节点数据域为'*'
    root->weight = lt->weight + rt->weight;    // 根节点的权值为左右孩子权值之和
    root->lchild = lt;   root->rchild = rt;    // 接上左右子树
    Insert(H, root);                                   // 将组合成的新树插入堆
  }  // 每次合二为一减少了一棵树，while循环结束时，优先队列只含的一棵树即为所求
  return root;                          // 返回构造的赫夫曼树
}
```



清華大學出版社 TSINGHUA UNIVERSITY PRESS

堆的两个操作要点：

- DelTop(H) 可能发生一次向下调整过程；
- Insert(H, root) 可能发生一次向上调整过程。

以下简化演示过程

建赫夫曼树

```
HFTNode *MakeHFTree(char data[], int weight[], int n) { //算法6.15 构造赫夫曼树
  // 由长度为n的字符数组data和权值数组weight构造并返回赫夫曼树，使用小顶堆存储森林
  ... ...  /* 建小顶堆H */
  while(H->len > 1) {                    // while循环每次将权值最小的两棵树合二为一
    lt = DelTop(H);   rt = DelTop(H);          // 从堆取权值最小两棵树为左、右子树
    if (!(root=(HFTNode*)malloc(sizeof(HFTNode)))) exit(OVERFLOW); // 根节点
    root->data = '*';                                  // 内部节点数据域为'*'
    root->weight = lt->weight + rt->weight;    // 根节点的权值为左右孩子权值之和
    root->lchild = lt;   root->rchild = rt;    // 接上左右子树
    Insert(H, root);                                   // 将组合成的新树插入堆
  }  // 每次合二为一减少了一棵树，while循环结束时，优先队列只含的一棵树即为所求
  return root;                          // 返回构造的赫夫曼树
}
```

6.6 树和森林

6.6.1 树的存储结构

1. 双亲表示法

假设以一组连续空间存储树的节点，同时在每个节点中附设一个指示器指示其双亲节点在链表中的位置，其形式说明如下：

```
typedef struct PTNode {
    TElemType  data;      // 数据元素
    int        parent;    // 双亲位置域
} PTNode;                 // 节点类型
typedef struct {
    PTNode  *nodes;       // 节点动态数组
    int      size;        // nodes的容量
    int      n, r;        // 节点数和根节点的位置
} *PTree;                 // 树的双亲表指针类型
```

例如，图6.22展示一棵树及其双亲表示的存储结构。

利用了每个节点（除根以外）只有唯一双亲的性质，可以直接找到节点的双亲，但是求节点的孩子时，却需要一个遍历的过程。

双亲表只用于特定需求，比如，在并查集可看到它的应用。

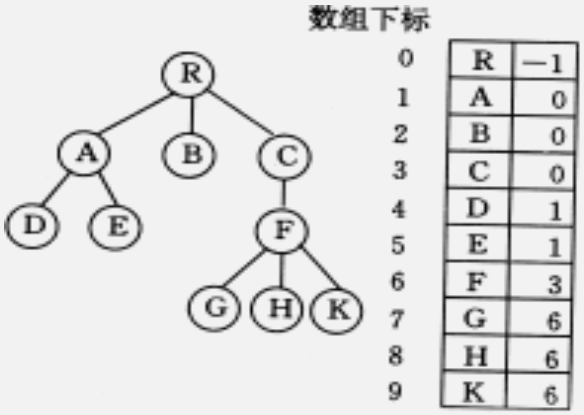


图6.22　树的双亲表示法示例

6.6.1 树的存储结构（续）

2．孩子表示法

把每个节点的孩子节点排列起来，看成是一个**线性表**，且以**单链表**作存储结构，这样n个节点就有n个**孩子链表**（叶子的孩子链表为空表）。而n个头指针又组成一个线性表，为了便于查找，可采用**顺序存储结构**。这种存储结构称为孩子链表，

与双亲表示法相反，孩子表示法便于那些涉及**孩子**和**子孙**的操作，却不适用于求父节点和祖先的操作。

取长补短，可把双亲表示法和孩子表示法结合，将**双亲表和孩子链表合而为一**，也就是在孩子链表的树节点增加一个**parent**域，值为双亲节点的下标。图6.23(b)是这种存储结构的一例，和(a)表示同一棵树。

```
typedef struct CTNode {
    int      child;      // 孩子节点在nodes数组的下标
    CTNode *next;        // 链域
} *ChildPtr;             // 孩子节点指针类型
typedef struct {
    TElemType  data;
    ChildPtr   firstChild;   // 孩子链树头指针
} CTBox;                     // 树节点类型
typedef struct {
    CTBox  *nodes;           // 节点动态数组
    int     n, r;            // 节点数和根节点的位置
    int     size;            // 容量
} *CTree;                    // 孩子链表树指针类型
```

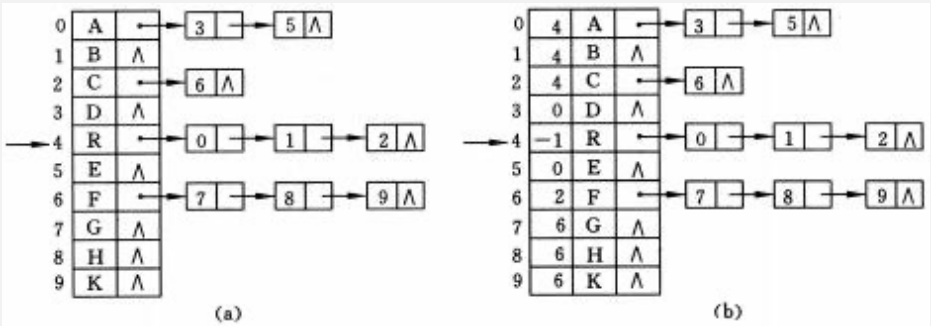


图6.23 图6.22的树的另外两种表示法
(a) 孩子链表； (b) 带双亲的孩子链表

6.6.1 树的存储结构（续）

3．孩子兄弟表示法

又称二叉树表示法，或二叉链表表示法。即以二叉链表作树的存储结构。链表中节点的两个链域分别指向该节点的第一个孩子节点和下一个兄弟节点，分别命名为firstchild 域和nextsib域。

```
typedef struct CSNode {
    TElemType  data;       // 数据元素
    CSNode   *firstChild, // 第一孩子指针
             *nextSib;     // 下一兄弟指针
} CSNode, *CSTree;    // 节点类型，孩子兄弟链表指针类型
```

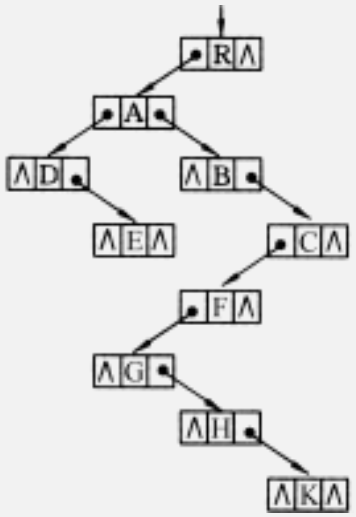


图6.24　图6.22中树的二叉链表表示法

图6.24是图6.22中树的孩子兄弟链表。这种存储结构便于实现各种树的操作，易于实现找节点孩子等的操作。

例如：若要访问节点x的第i个孩子，则只要先从firstChild域找到第1个孩子节点，然后沿着孩子节点的nextSib域连续走i-1步，便可找到x的第i个孩子。

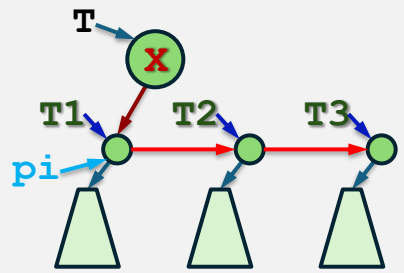
当然，如果为每个节点增设一个parent域，则同样能方便地实现找双亲的操作。

树的基本操作

执行：

```
CSTree newT = MakeTree('X', 3, T1, T2, T3);
```

（1）**组建树**。**算法6.17**组建一棵根节点值为e且有*n*棵子树的树。由于无法实现确定子树的数目，所以采用第5章中*n*维数组采用过的变长参数表，以获取*n*棵子树的实参。



argptr

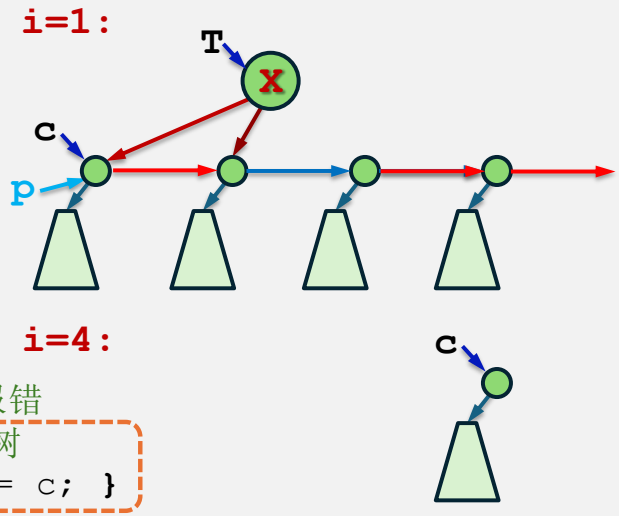
```
CSTree MakeTree(TElemType e, int n, ...) { // 算法 6.17
   // 组建根节点e和n棵子树的树，变长参数为n棵子树
   int i;   CSTree T, pi;
   va_list argptr;          // argptr是存放变长参数表信息的数组  va_list用void*代替
   if (!(T = (CSTree)malloc(sizeof(CSNode)))) exit(OVERFLOW);
   T->data = e;                  // 根节点的值为e
   T->firstChild = T->nextsib = NULL;
   if (n<=0) return T;           // 若无子树，则返回仅含根节点的单节点树
   va_start(argptr, n);          // 令argptr指向参数n后的第一个实参（第1棵子树根指针）
   T->firstChild = pi = va_arg(argptr, CSTree); // 取第1棵子树实参并转换为CSTree类型
   for (i=1; i<n; i++) {                        // 将n棵树作为根节点的子树插入
      pi->nextsib = va_arg(argptr, CSTree);     // 取下一棵子树实参并转换为CSTree类型
      pi = pi->nextsib;
   }
   va_end(argptr);          // 取实参结束
   return T;                // 返回组建的树
}
```

孩子兄弟链表存储的树像一棵“歪脖子树”

树的基本操作（续）

（2）**插入第i棵子树**。**算法6.18**将树c插入为树T的第i棵子树。

当树非空时，若i==1，则树c直接作为第一棵子树插入；否则，先确定第i-1棵子树的位置，然后将c子树插入其后。




```
Status InsertChild(CSTree T, int i, CSTree c) {
   // 算法 6.18。插入c为T的第i棵子树，c非空且与T不相交
   int j;   CSTree p;
   if (!T || i<1) return ERROR;   // T是空树或i<1，则报错
   if (1==i)                      // c插入为T的第1棵子树
      { c->nextsib = T->firstChild;  T->firstChild = c; }
   else {
      p = T->firstChild;          // p指向T的第1棵子树
      for (j=2; p!=NULL && j<i; j++) p = p->nextsib;   // 寻找插入位置
      if (j == i)                                      // 找到插入位置
         { c->nextsib = p->nextsib;   p->nextsib = c; }
      else return ERROR;    // 插入位置i过大，  报错
   }
   return OK;
}
```

如果基于一种存储结构实现树的一组基本操作，就得到了树的一个数据类型，可支持树相关的应用程序开发。

6.6.2 森林与二叉树的转换

可以二叉链表作为媒介导出树与二叉树之间的一个**对应关系**：给定一棵树，可以找到唯一的一棵二叉树与之对应，从物理结构来看，它们的**二叉链表是相同**的，只是**解释不同**而已。

图6.25直观地展示了树与二叉树之间的**对应关系**。

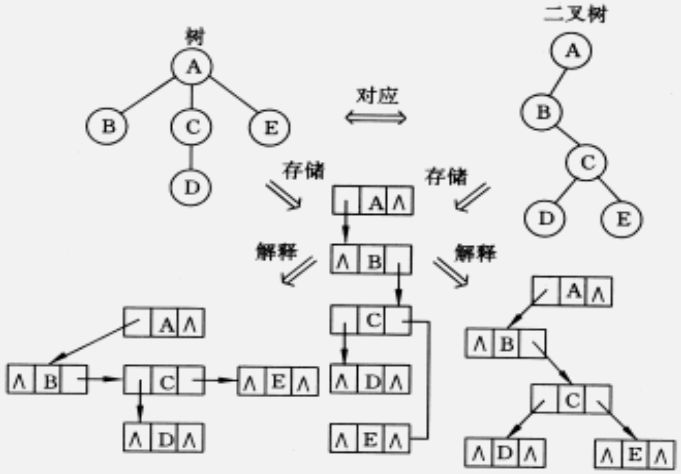
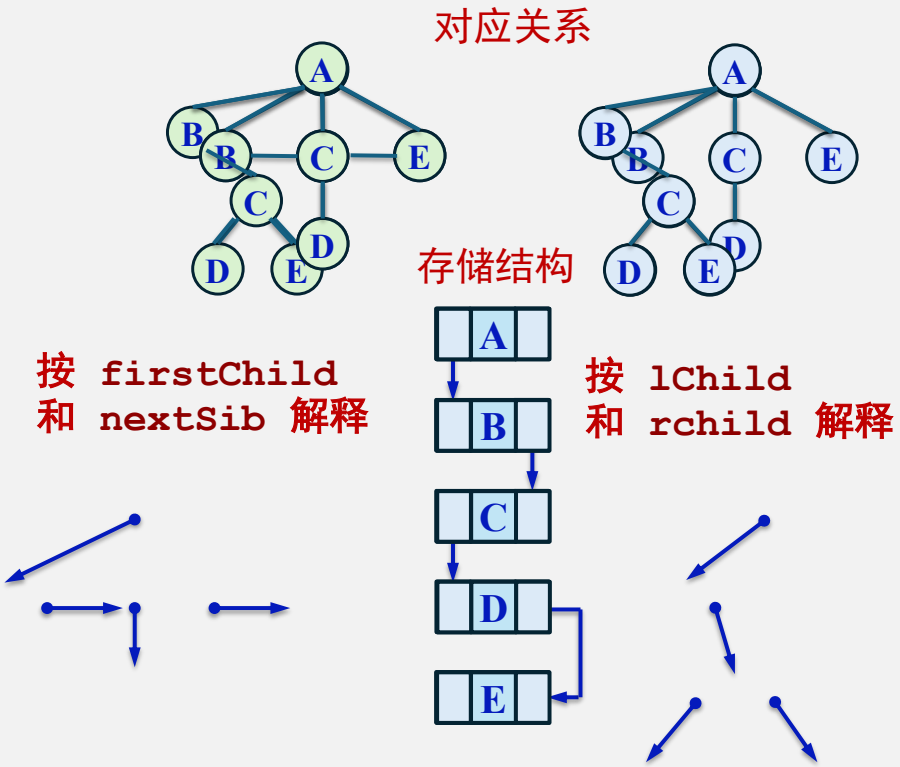


图6.25 树与二叉树的对应关系示例



6.6.2 森林与二叉树的转换（续）

森林与二叉树之间的对应关系

从树的二叉链表表示的定义可知，任何一棵和树对应的二叉树，其**右子树必空**。

若把森林中第二棵树的根节点看成是第一棵树的根节点的兄弟，则同样可导出**森林和二叉树的对应关系**。

例如，图6.26展示了森林与二叉树之间的对应关系。

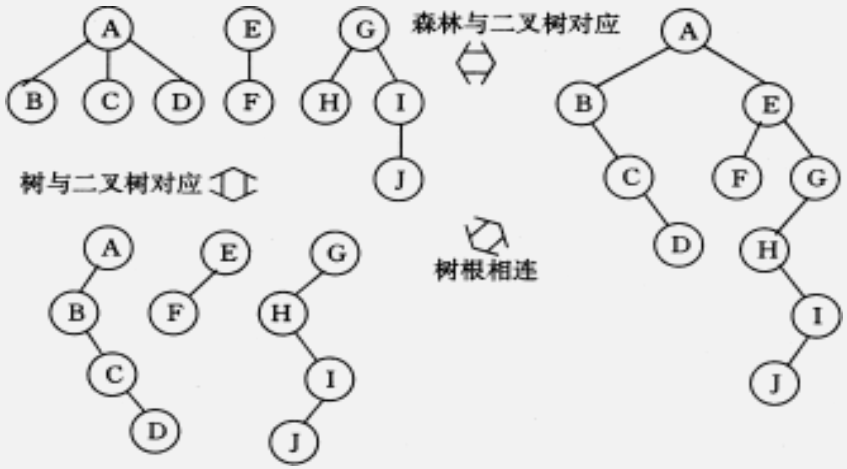


图6.26 森林与二叉树的对应关系示例

如教材所述，这个一一对应关系导致森林或树与二叉树可以**相互转换的形式定义。**

6.6.3 树和森林的遍历

按照森林和树相互递归的定义，可以推出**森林**的两种遍历方法：

1. 先序遍历森林

若森林非空，则按下述规则遍历之：

（1）访问森林中第一棵树的根节点；

（2）先序遍历第一棵树中根节点的子树森林；

（3）先序遍历除去第一棵树之后剩余的树构成的森林。

2. 中序遍历森林

若森林非空，则按下述规则遍历之：

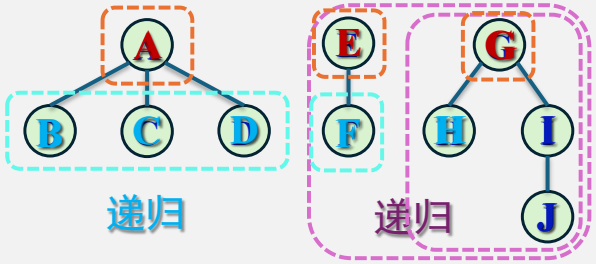
（1）中序遍历森林中第一棵树的根节点的子树森林；

（2）访问第一棵树的根节点；

（3）中序遍历除去第一棵树之后剩余的树构成的森林。

先序遍历：

图6.26的森林



中序遍历：

由森林与二叉树之间的**转换规则**可知：

树的遍历和二叉树遍历的对应关系为

树	森林	二叉树
先根遍历	先序遍历	先序遍历
后根遍历	后序遍历	中序遍历

6.7 并查集与等价问题

6.7.1 等价关系和等价类

如果集合S中的关系R满足如下性质，则称它为一个**等价关系**。

- **自反性**：对于任意元素$x \in S$，xRx为真；
- **对称性**：对于任意两个元素$x, y \in S$，如果xRy为真，则yRx也为真；
- **传递性**：对于任意三个元素$x, y, z \in S$，如果xRy和yRz都为真，则xRz也为真。

例如，一个整数集合上的关系≤（小于或等于）和≥（大于或等于）就不是等价关系。它们具有**自反性**（$x \leqslant x$）和传递性（$x \leqslant y$，$y \leqslant z$，则$x \leqslant z$），但不满足对称性（$x \leqslant y$成立时，却$y \leqslant x$不一定成立）。

双向网络连接是一种等价关系。首先它是**自反的**，因为为何节点都连接自身。如果节点x连接y，那么y也连接x，具有**对称性**。如果节点x连接y，且y连接z，那么x也连接z。

设R是集合S的等价关系。对任何$x \in S$，由 $[x]_R = \{ y \mid y \in S \wedge x\ R\ y \}$给出的集合$[x]_R \subseteq S$称为由$x \in S$生成的一个$R$等价类。

如果R是集合S上的一个等价关系，则由这个等价关系可产生这个集合的唯一划分。也就是可以按R将S划分为若干不相交的子集$S_1, S_2, \ldots, S_n$，它们的并集为S，而这些子集S_i，便称为S的R**等价类**。

等价关系是现实世界中广泛存在的一种关系，许多应用问题可以归结为按给定的等价关系划分某集合为等价类，通常称这类问题为**等价问题**。

6.7.1 等价关系和等价类（续）

应如何划分等价类呢？假设集合S有n个元素，m个形如(x, y)（$x,y\in S$）的等价偶对确定了等价关系R，需求S的划分。

确定等价类的算法：

（1）令S中每个元素各自形成一个只含单个成员的子集，记作$S_1, S_2, \ldots, S_n$。

（2）重复读入m个偶对，对每个读入的偶对(x, y)，判定x和y所属子集。不失一般性，假设$x\in S_i, y\in S_j$，若$S_i \neq S_j$，则将S_i并入S_j，并置S_i为空（或反过来，将S_j并入S_i，并置S_j为空）。当m个偶对都被处理过后，$S_1, S_2, \ldots, S_n$中所有非空子集即为S的R等价类。



6.7.2 并查集的定义和实现

划分等价类需对集合进行的操作有3个：

1. **创建**n个等价类——构造含n个成员的集合；
2. **查找**等价类——查（Find）某个单元素所在子集；
3. **合并**等价类——并（Union）两个互不相交的子集并为一个子集。

由此，这样的集合也称为**并查集**，可定义为包含上述三种操作的抽象数据类型UFSet。

ADT UFSet {

数据对象：若设S是UFSet类型的集合，则它由n(n>0）个子集S_i(i=1,2,…,n）构成，
每个子集的成员都是子界［1..MaxNumber］内的整数

数据关系：$S_1 \cup S_2 \cup \ldots \cup S_n = S \quad S_i \subset S(i=1,2,\ldots,n)$

基本操作：

Initial(n,x_1,x_2,…,x_n);
操作结果：初始化操作。构造并返回一个由n个子集（每个子集只含单个成员x_i）构成的集合S。

Find(S, x);
初始条件：S是已存在的集合，x是S中某个子集的成员。
操作结果：查找操作。确定s中x所属子集S_i。

Union(S, i, j):
初始条件：S_i和S_j，是S中的两个互不相交的非空集合。
操作结果：归并操作。将S_i和S_j中的一个并入另一个中。

} ADT UFSet;

6.7.2 并查集的定义和实现（续）

为高效实现并查集的查找和合并这两种操作，可采用树形结构来表示集合，树的每个结点表示集合中的一个元素，结点的存储单元下标来指代集合成员，避免频繁用$O(n)$的时间代价查找结点（这是基于结点的存储单元下标来指代结点元素的这种方式，比如图的顶点用序号或下标来指代）。由于一棵树只有一个树根结点，故多棵树的并查集森林可以忽略对并查集森林中各子集树根位置的管理。需要时，调用查找操作来确定某个结点所在子集树的根结点的位置，或者通过遍历获得所有子集树的根。

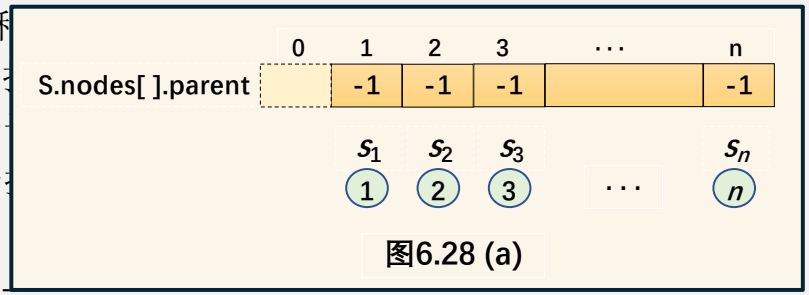


图6.28 (a)

```
//------ ADT UFSet采用树的双亲表存储表示 -----
typedef PTree  UFSet;
```

并查集的初始化如算法6.19表述。

```
UFSet InitUFSet(char ds[], int n) {  // 算法6.19，初始化并查集S
   UFSet S;
   if (!(S = (UFSet)malloc(sizeof(*S)))) exit(OVERFLOW);    // 分配结构记录
   if (!(S->nodes = (PTNode*)calloc(n+1, sizeof(PTNode)))) exit(OVERFLOW);
   for (int i=1; i<=n; i++) {
      S->nodes[i].data = ds[i-1];   // 保存成员值
      S->nodes[i].parent = -1;      // 每个成员自成一棵子集树，根的双亲为-1
   }
   S->n = n;    return S;            // 返回构造的初始并查集
}
```

含n个元素的并查集{1,2,3,…,n}的初始状态如图6.28(a)所示。



6.7.2 并查集的定义和实现（续）

查找函数Fine和合并操作Union的实现如算法6.20和算法6.21所示。

```
int Find(UFSet S, int i) {  // 算法 6.20，找并查集S中i所在子集的根
    if (i<1 || i>S->n) return 0;  // i不属S中任一子集
    while (S->nodes[i].parent > 0) i = S->nodes[i].parent;
    return i;
}
Status Union(UFSet S, int i, int j){  // 算法 6.21，合并Sj=SiUSj
    if (i<1 || i>S->n || j<1 || j>S->n) return ERROR;
    if (S->nodes[i].parent > 0) i = Find(S, i);  // 确保i指示其所在子集的根
    if (S->nodes[j].parent > 0) j = Find(S, j);  // 确保j指示其所在子集的根
    if (i != j) S->nodes[i].parent = j;          // 若i和j分属不同子集，则Si并入Sj
    return OK;
}
```

算法6.20和算法6.21的时间复杂度分别为$O(d)$和$O(1)$，其中d是树的深度。

清華大學出版社 TSINGHUA UNIVERSITY PRESS

6.7.3 对“并”算法的改进——加权合并法

改进的办法称为**加权合并法**（Weight Union Rule）：先判别子集中所含成员的数目，然后令含成员少的子集树根节点指向含成员多的子集的根。

为此，需相应地修改存储结构：令根节点的parent域存储子集中所含成员数目的负值。修改后的“并”操作算法如**算法6.22**所示。

```
Status Union_wt(UFSet S, int i, int j) {  // 算法 6.22,
  if (i<1 || i>S->n ||j<1 || j>S->n) return ERROR;
  if (S->nodes[i].parent > 0) i = Find(S, i);     // 确
  if (S->nodes[j].parent > 0) j = Find(S, j);     // 确
  if (i != j)                                       //
    if (S->nodes[i].parent > S->nodes[j].parent) { //
      S->nodes[j].parent += S->nodes[i].parent;     //
      S->nodes[i].parent = j;                  // 置Sj的根
    } else {                                   // Si的成员
      S->nodes[i].parent += S->nodes[j].parent;
      S->nodes[j].parent = i;
    }
  return OK;
}
```

例6-2 假设集合S={ x | 1≤x≤n是正整数}，*R*是*S*上的一个等价关系。

R = { (1,2), (3,4), (5,6), (7,8), (1,3), (5,7), (1,5), … }

现求*S*的等价类。

S.nodes[].parent

	0	
Union_wt(S,1,2)	1	-8
Union_wt(S,3,4)	2	1
Union_wt(S,5,6)	3	1
Union_wt(S,7,8)	4	3
Union_wt(S,1,3)	5	1
Union_wt(S,5,7)	6	5
Union_wt(S,1,5)	7	5
	8	7
	9	-1
	⋮	⋮
	n	-1

6.7.4 对“查”算法的改进——路径压缩法

如例6-2所示，随着子集逐对合并，树的深度也越来越大。

为了进一步减少确定元素所在集合的时间，还可进一步将算法6.20改进为**算法6.23**。

当所查元素*i*不在树的第二层时，在查找算法中增加一个“**压缩路径**”（Path Compression）功能，将所有从元素*i*到根路径上的元素都变成树根的孩子。

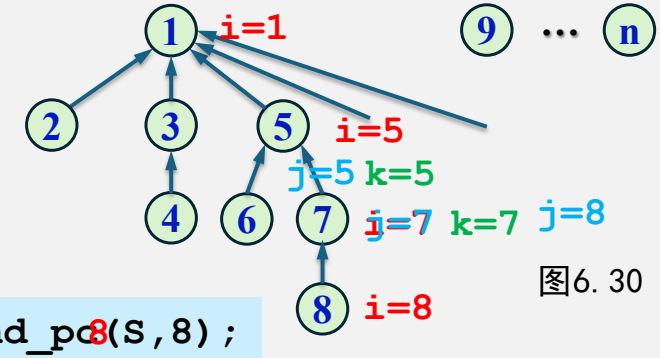


图6.30

执行：int si = Find_pc(S,8);

```
int Find_pc(UFSet S, int i) {   // 算法 6.23，查i所属子集的根，并压缩从i到根的路径
    if (i<1 || i>S->n) return 0;
    if (S->nodes[i].parent<0) return i;
    int j = i;
    while (S->nodes[i].parent > 0) i = S->n
    while (S->nodes[j].parent != i) {
        int k = S->nodes[j].parent;
        S->nodes[j].parent = i;
        j = k;
    }
    return i;
}
```

在Union_wt中，可用Find_pc取代Fine。

已经证明，利用算法Find_pc和Union_wt划分大小为n的集合为等价类的**时间复杂度**为$O(n\alpha(n))$。其中$\alpha(n)$是一个增长极其缓慢的函数，若定义单变量的阿克曼函数为$A(x) = A(x, x)$，则函数$\alpha(n)$定义为$A(x)$的拟逆，即$\alpha(n)$的值是使$A(x) \geq n$成立的最小x。所以，对于通常所见到的正整数n而言，$\alpha(n) \leq 4$。

6.8 回溯法与树的遍历

回溯法是设计递归过程的一种重要方法，它的求解过程实质上是一个**先序遍历**一棵“**状态树**”的过程，只是这棵树不是遍历前预先建立的，而是隐含在遍历过程中，但如果认识到这点，很多问题的递归过程设计也就迎刃而解。

例6-3 求含n个元素的集合的幂集。集合A的幂集是由集合A的所有子集所组成的集合。如：$A=(1,2,3)$，则A的幂集为

$$\rho(A)=\{\{1,2,3\},\{1,2\},\{1,3\},\{1\},\{2,3\},\{2\},\{3\},\{\}\} \quad (6\text{-}6)$$

除了可用5.6节介绍的**分治法**来设计这个求幂集的递归过程，还可以从另一角度分析问题：

幂集的每个元素是一个集合，它或是空集，或含集合A中一个元素，或含集合A中多个元素，或等于集合A。
反之，从集合A的每个元素来看，它只有两种状态：它或属幂集的元素集，或不属幂集元素集。

这样，求幂集$\rho(A)$的元素的过程可看成是依次对集合A中元素进行“取”或“舍（弃）”的过程，并且可以用一棵如图6.31所示的二叉树来表示过程中幂集元素的状态变化状况。

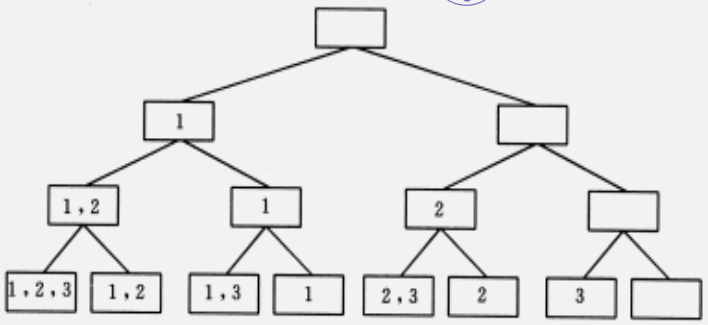


图6.31 幂集元素在生成过程中的状态图

树中的根节点表示幂集元素的初始状态（为空集）；叶子节点表示它的终结状态（如图6.31中8个叶子节点表示式（6-6）中幂集$\rho(A)$的8个元素）；而第i（$i=2,3,\ldots,n$-1）层的分支节点，则表示已对集合A中前i-1个元素进行了取/舍处理的当前状态（左分支表示“取”，右分支表示“舍”）。
因此求幂集元素的过程即为先序遍历这棵状态树的过程，可由**算法6.25**实现。

建议在AnyviewC可视跟踪算法6.25产生和输出所求幂集的过程